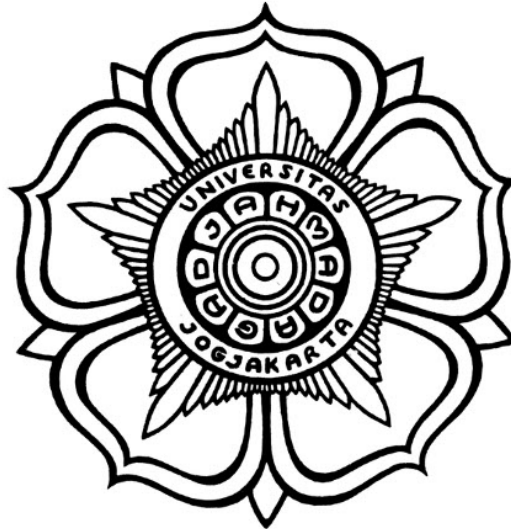


**MANAJEMEN PERANGKAT LUNAK STANDAR ISO27001/2022
SECURE CODING UNTUK PENINGKATAN DAN KONVERSI
VERSI WEB APP DENGAN MEMANFAATKAN AI DAN
PERANGKAT OPENSOURCE.**

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Affordable and Clean Energy
Climate Action

Disusun oleh:

Muhammad Farrel Akbar
22/492806/TK/53947

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
«TAHUN PENDADARAN»**

HALAMAN PENGESAHAN

MANAJEMEN PERANGKAT LUNAK STANDAR ISO27001/2022 SECURE CODING UNTUK PENINGKATAN DAN KONVERSI VERSI WEB APP DENGAN MEMANFAATKAN AI DAN PERANGKAT OPENSOURCE.

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Muhammad Farrel Akbar
22/492806/TK/53947

Telah disetujui dan disahkan

Pada tanggal

Dosen Pembimbing I

Dosen Pembimbing II

Dani Adhipta, S.Si., M.T.
«NIP xxxxxx»

Addin Suwastono, Ir., S.T., M.Eng., IPM.
«NIP xxxxxx»

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama : Muhammad Farrel Akbar
NIM : 22/492806/TK/53947
Tahun terdaftar : 2022
Program : Sarjana
Program Studi : Teknologi Informasi
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, tanggal-bulan-tahun

Materai Rp10.000

(Tanda tangan)

Muhammad Farrel Akbar
22/492806/TK/53947

HALAMAN PERSEMBAHAN

Tugas akhir ini saya persembahkan kepada kedua orang tua saya yang telah mendukung dan mendoakan saya hingga titik ini. Saya persembahkan pula kepada keluarga dan teman-teman semua, serta untuk bangsa, negara, dan agamaku.

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Dani Adhipta, S.Si., M.T. yang telah membimbing dan memberikan arahan serta ilmu yang sangat berharga selama penyusunan skripsi ini.
2. Addin Suwastono, Ir., S.T., M.Eng., IPM. yang telah membimbing dan memberikan arahan serta ilmu yang sangat berharga selama penyusunan skripsi ini.
3. Orang tua saya, Bapak Andrian Nugroho dan Ibu Retny Anggeriana yang telah memberikan dukungan moril dan materiil serta doa yang tiada henti sehingga saya dapat menyelesaikan studi ini.

Saya sebagai penulis menyadari bahwa dalam penyusunan skripsi ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi kesempurnaan skripsi ini. Semoga skripsi ini dapat bermanfaat bagi pembaca pada umumnya dan bagi penulis pada khususnya. Amin.

Yogyakarta, 13 Mei 2026

Muhammad Farrel Akbar
22/492806/TK/53947

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	v
DAFTAR ISI	vi
DAFTAR TABEL	ix
DAFTAR GAMBAR	x
DAFTAR SINGKATAN.....	xi
INTISARI.....	xiii
ABSTRACT	xiv
BAB I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	4
1.4 Batasan Penelitian	5
1.5 Manfaat Penelitian	6
1.5.1 Manfaat Akademis	6
1.5.2 Manfaat Praktis.....	6
1.6 Sistematika Penulisan.....	6
BAB II Tinjauan Pustaka dan Dasar Teori	8
2.1 Tinjauan Literatur	8
2.1.1 Perlindungan Otomatis Aplikasi PHP dari Injeksi SQL.....	8
2.1.2 Menggunakan LLM untuk Menemukan dan Memantau Kerentanan	8
2.1.3 Transformasi Kode dengan Bantuan LLM	9
2.1.4 Pendekatan Neuro-Symbolik untuk Deteksi Kerentanan Keamanan	9
2.1.5 Dampak Jangka Panjang Transformasi Kode Otomatis	10
2.1.6 Transformasi Kode Berbasis Pola untuk Migrasi Aplikasi.....	10
2.1.7 Migrasi Kode Berbasis AST pada Skala Industri	11
2.1.8 Ringkasan Perbandingan.....	12
2.2 Dasar Teori	13
2.2.1 PHP dan Perubahan Antar Versi	13
2.2.2 ISO/IEC 27001:2022 dan <i>Secure Coding</i>	14
2.2.3 OWASP Top 10 dan Kerentanan Aplikasi Web PHP	14
2.2.4 DevSecOps dan Integrasi Keamanan dalam <i>Pipeline</i> Pengembangan	15
2.2.5 <i>Abstract Syntax Trees</i> dan <i>Static Analysis</i>	15
2.2.6 Rector: Alat Transformasi Kode Berbasis AST	16

2.2.7	PHPStan: Alat Analisis Statis untuk PHP	16
2.2.8	Semgrep: Alat Analisis Statis Ringan untuk Keamanan	16
2.2.9	Model-Model LLM untuk Analisis Kode	17
2.2.10	Ollama: Menerapkan LLM Secara Lokal	17
BAB III Metodologi Penelitian		19
3.1	Alat dan Bahan Penelitian	19
3.1.1	Perangkat Keras	19
3.1.2	Perangkat Lunak	19
3.1.3	Model LLM yang Dievaluasi	20
3.1.4	Bahan Penelitian	21
3.2	Metode Penelitian	21
3.3	Arsitektur Sistem	23
3.3.1	Scanner (scanner.py)	23
3.3.2	Converter (converter.py)	24
3.3.3	Analyzer (analyzer.py)	24
3.3.4	AI Engine (ai_engine.py)	24
3.3.5	ISO Mapper (iso_mapper.py)	25
3.3.6	Main (main.py)	25
3.3.7	Composer Analyzer (composer_analyzer.py)	25
3.4	Alur Data Pipeline	26
3.5	Desain Prompt LLM	27
3.5.1	Struktur Prompt	27
3.5.2	Keputusan Desain Prompt	27
3.6	Desain Pemetaan ISO/IEC 27001:2022	28
3.6.1	Mekanisme Klasifikasi Temuan	29
3.7	Dataset Studi Kasus	30
3.8	Rancangan Eksperimen Komparasi Model LLM	30
3.8.1	Kondisi Eksperimen	30
3.8.2	Metrik Evaluasi	30
3.8.3	Interpretasi Trade-off Antar Metrik	32
3.9	Keamanan dan Privasi Data	32
3.10	Metode Pengujian dan Evaluasi	32
BAB IV Hasil dan Diskusi		34
4.1	Hasil Konversi Rector	34
4.1.1	Contoh Transformasi Rector	35
4.1.2	File yang Gagal Dikonversi	36
4.2	Hasil Pemindaian Keamanan Semgrep	37
4.2.1	Pre-Scan: Baseline Keamanan Kode Asli	37
4.2.2	Post-Scan: Perbandingan Setelah Konversi	37

4.3	Hasil Validasi PHPStan	38
4.4	Status Kepatuhan ISO/IEC 27001:2022	39
4.5	Hasil Komparasi Lima Model LLM (Kondisi A)	40
4.5.1	Format Compliance Rate	40
4.5.2	Syntax Validity Rate	41
4.5.3	Statistik Ringkasan.....	42
4.5.4	Analisis Kualitatif: Perbandingan Respons terhadap Temuan SQL Injection	42
4.5.5	Trade-off dan Implikasinya	44
4.6	Hasil Komparasi Lima Model LLM (Kondisi B).....	44
4.6.1	Format Compliance Rate	45
4.6.2	Syntax Validity Rate	46
4.6.3	Statistik Ringkasan.....	47
4.7	Perbandingan Kondisi A dan Kondisi B	48
4.8	Optimasi Parameter Qwen2.5-Coder (Kondisi C).....	49
4.8.1	Pengaruh Temperature	49
4.8.2	Pengaruh Context Window	50
4.8.3	Implikasi Kondisi C.....	50
4.9	Analisis Dependensi Composer (FT UGM)	50
4.10	Diskusi dan Keterbatasan.....	51
4.10.1	Keterbatasan Dataset.....	51
4.10.2	Keterbatasan Teknis Pipeline	51
4.10.3	Interpretasi Confidence Score	52
4.11	Studi Kasus 2: Sistem CBT DTI UGM	52
4.11.1	Hasil Konversi Rector	52
4.11.2	Hasil Pemindaian Keamanan Semgrep	53
4.11.3	Hasil Validasi PHPStan	53
4.11.4	Status Kepatuhan ISO/IEC 27001:2022	54
4.11.5	Analisis Dependensi Composer	55
4.12	Perbandingan Kedua Studi Kasus	56
4.12.1	Perbandingan Temuan Keamanan Semgrep	56
4.12.2	Perbandingan Tingkat Konversi Rector	57
4.12.3	Perbandingan Temuan PHPStan.....	57
4.12.4	Ringkasan Perbandingan dan Analisis Dependensi Composer	58
BAB V	Kesimpulan dan Saran.....	60
5.1	Kesimpulan.....	60
5.2	Saran.....	61
DAFTAR PUSTAKA		63

DAFTAR TABEL

Tabel 2.1	Perbandingan Penelitian Terdahulu dan Karakteristik Penelitian	12
Tabel 2.2	Perbandingan Penelitian Terdahulu dan Aspek Metodologi.....	12
Tabel 3.1	Pemetaan argumen <code>-target-php</code> ke konfigurasi Rector	24
Tabel 3.2	Pemetaan sumber temuan ke kontrol ISO/IEC 27001:2022 Annex A	29
Tabel 3.3	Pemetaan kata kunci <code>rule_id</code> Semgrep ke jenis kerentanan dan kontrol ISO/IEC 27001:2022	29
Tabel 4.1	Status kepatuhan ISO/IEC 27001:2022 hasil <i>pipeline</i> pada dataset FT UGM	39
Tabel 4.2	Perbandingan metrik utama Kondisi A vs Kondisi B untuk kelima model LLM	48
Tabel 4.3	Status kompatibilitas PHP 8.x dependensi <i>composer</i> FT UGM.....	50
Tabel 4.4	Status kepatuhan ISO/IEC 27001:2022 hasil <i>pipeline</i> pada dataset CBT DTI UGM.....	55

DAFTAR GAMBAR

Gambar 3.1	Alur penelitian dengan model prototipe	22
Gambar 3.2	Arsitektur sistem <i>pipeline</i> dengan enam modul Python.....	23
Gambar 3.3	Alur data <i>pipeline</i> dari <i>input</i> hingga laporan akhir	26
Gambar 4.4	Ringkasan hasil konversi Rector terhadap 245 <i>file</i> PHP dataset FT UGM.....	34
Gambar 4.5	Distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 Annex A	38
Gambar 4.6	<i>Format compliance rate</i> kelima model LLM pada Kondisi A (<i>zero-shot</i> , <i>prompt</i> identik)	40
Gambar 4.7	<i>Syntax validity rate</i> dan <i>code block extraction rate</i> kelima model LLM pada Kondisi A	41
Gambar 4.8	Statistik lengkap komparasi kelima model LLM pada Kondisi A .	42
Gambar 4.9	<i>Format compliance rate</i> kelima model LLM pada Kondisi B (<i>prompt</i> dioptimasi per model)	45
Gambar 4.10	<i>Syntax validity rate</i> dan <i>code block extraction rate</i> kelima model LLM pada Kondisi B	46
Gambar 4.11	Statistik lengkap komparasi kelima model LLM pada Kondisi B .	47
Gambar 4.12	Perbandingan metrik Qwen2.5-Coder pada variasi <i>temperature</i> dan <i>context window</i> (Kondisi C)	49
Gambar 4.13	Ringkasan hasil konversi Rector terhadap 183 <i>file</i> PHP dataset CBT DTI UGM	53
Gambar 4.14	Distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 Annex A pada dataset CBT DTI UGM	54
Gambar 4.15	Perbandingan temuan Semgrep <i>pre-scan</i> berdasarkan jenis kerentanan pada kedua studi kasus.....	56
Gambar 4.16	Perbandingan hasil konversi Rector pada kedua studi kasus.....	57
Gambar 4.17	Perbandingan <i>error</i> PHPStan berdasarkan kontrol ISO/IEC 27001:2022 pada kedua studi kasus.....	58
Gambar 4.18	Ringkasan perbandingan metrik <i>pipeline</i> dan status dependensi <i>composer</i> kedua studi kasus	58

DAFTAR SINGKATAN

DAFTAR SINGKATAN DAN SIMBOL

CR	=	<i>Conversion Rate</i> (tingkat konversi)
FCR	=	<i>Format Compliance Rate</i>
SVR	=	<i>Syntax Validity Rate</i>
ΔF	=	<i>Security Finding Delta</i> (selisih temuan keamanan)
$\bar{C}_m$	=	<i>Mean Confidence Score</i> model m
σ_m^2	=	variansi <i>confidence score</i> model m
AI	=	<i>Artificial Intelligence</i>
API	=	<i>Application Programming Interface</i>
AST	=	<i>Abstract Syntax Tree</i>
CVE	=	<i>Common Vulnerabilities and Exposures</i>
DTI	=	Direktorat Teknologi Informasi
FT	=	Fakultas Teknik
GPU	=	<i>Graphics Processing Unit</i>
IEC	=	<i>International Electrotechnical Commission</i>
ISMS	=	<i>Information Security Management System</i>
ISO	=	<i>International Organization for Standardization</i>
JIT	=	<i>Just-In-Time</i>
LLM	=	<i>Large Language Model</i>
MVC	=	<i>Model-View-Controller</i>
NVD	=	<i>National Vulnerability Database</i>
OWASP	=	<i>Open Web Application Security Project</i>
PDO	=	<i>PHP Data Objects</i>
PHP	=	<i>PHP: Hypertext Preprocessor</i>
REST	=	<i>Representational State Transfer</i>
SAST	=	<i>Static Application Security Testing</i>
SPL	=	<i>Software Product Line</i>
SQL	=	<i>Structured Query Language</i>
SSRF	=	<i>Server-Side Request Forgery</i>
UGM	=	Universitas Gadjah Mada
VRAM	=	<i>Video Random Access Memory</i>
XSS	=	<i>Cross-Site Scripting</i>
YAML	=	<i>YAML Ain't Markup Language</i>

INTISARI

Intisari ditulis menggunakan bahasa Indonesia dengan jarak antar baris 1 spasi dan maksimal 1 halaman. Intisari sekurang-kurangnya berisi tentang latar belakang dan tujuan penelitian, metodologi yang digunakan, hasil penelitian, kesimpulan dan implikasi, dan Kata kunci yang berhubungan dengan penelitian.

Kata Kunci ditulis maksimal 5 kata yang paling berhubungan dengan isi skripsi. Silakan mengacu pada ACM / IEEE *Computing classification* jika Anda adalah mahasiswa Sarjana TI <http://www.acm.org/about/class/> atau mengacu kepada IEEE keywords http://www.ieee.org/documents/taxonomy_v101.pdf jika Anda berasal dari Prodi Sarjana TE.

Kata kunci : Kata kunci 1, Kata kunci 2, Kata kunci 3, Kata kunci 4, Kata kunci 5

Contoh Abstrak Teknik Elektro:

"Penelitian ini bertujuan untuk mengembangkan sistem pengendalian suhu ruangan dengan menggunakan microcontroller. Metodologi yang digunakan adalah desain sirkuit, implementasi sistem pengendalian, dan pengujian performa. Hasil penelitian menunjukkan bahwa sistem pengendalian suhu ruangan yang dikembangkan mampu mengendalikan suhu ruangan dengan akurasi sebesar $\pm 0,5^{\circ}\text{C}$. Kesimpulan dari penelitian ini adalah sistem pengendalian suhu ruangan yang dikembangkan efektif dan efisien.

Kata kunci: microcontroller, sistem pengendalian suhu, akurasi."

Contoh Abstrak Teknik Biomedis:

"Penelitian ini bertujuan untuk mengevaluasi keefektifan prototipe alat pemantau denyut jantung berbasis elektrokardiogram (ECG) untuk pasien jantung. Metodologi yang digunakan meliputi desain dan pembuatan prototipe, pengujian dengan pasien, dan analisis data. Hasil penelitian menunjukkan bahwa prototipe alat pemantau denyut jantung berbasis ECG memiliki akurasi yang baik dan mampu memantau denyut jantung pasien secara efektif. Kesimpulan dari penelitian ini adalah prototipe alat pemantau denyut jantung berbasis ECG merupakan solusi yang efektif dan efisien untuk memantau pasien jantung.

Kata kunci: elektrokardiogram, alat pemantau denyut jantung, akurasi."

Contoh Abstrak Teknologi Informasi:

"Penelitian ini bertujuan untuk mengevaluasi keamanan dan privasi pengguna aplikasi media sosial terpopuler. Metodologi yang digunakan meliputi analisis kebijakan privasi dan pengaturan keamanan, pengujian penetrasi, dan survei pengguna. Hasil penelitian menunjukkan bahwa beberapa aplikasi media sosial memiliki kebijakan privasi yang kurang jelas dan rendahnya tingkat keamanan. Kesimpulan dari penelitian ini adalah pentingnya meningkatkan kebijakan privasi dan tingkat keamanan pada aplikasi media sosial untuk melindungi privasi dan data pengguna.

Kata kunci: media sosial, keamanan, privasi, pengguna."

ABSTRACT

Abstract ditulis italic (miring) menggunakan bahasa Inggris dengan jarak antar baris 1 spasi dan maksimal 1 halaman. Abstract adalah versi Bahasa Inggris dari intisari. Abstract dapat ditulis dalam beberapa paragraf. Baris pertama paragraph harus menjorok ke dalam sekitar 1 cm. Tidak dsarankan menggunakan mesin penerjemah melainkan tulis ulang.

Keywords : Keyword 1, Keyword 2, Keyword 3, Keyword 4, Keyword 5

BAB I

PENDAHULUAN

1.1 Latar Belakang

PHP bukan bahasa yang didesain untuk menjadi bahasa pemrograman besar. Rasmus Lerdorf memperkenalkannya pada tahun 1995 sebagai kumpulan skrip sederhana untuk mengelola halaman web pribadinya [1]. Tapi dalam waktu beberapa tahun, PHP berkembang jauh melampaui tujuan awalnya, dan pada akhir 1990-an sudah menjadi pilihan dominan untuk pengembangan sisi *server* [1]. Kemudahan penggunaan, dukungan *hosting* yang luas, dan ekosistem pustaka yang terus bertumbuh menjadikannya pilihan yang wajar bagi banyak pengembang web, mulai dari individu hingga tim teknologi di institusi besar [1].

Data terbaru menegaskan dominasi ini. Menurut W3Techs per 8 April 2026, sebesar 71,7% dari seluruh situs web yang memiliki bahasa pemrograman sisi *server* yang diketahui menggunakan PHP [2]. Angka itu menempatkan PHP jauh di atas bahasa-bahasa lainnya. Namun di balik angka yang terlihat impresif tersebut ada kenyataan yang kurang nyaman: hanya 58,1% dari situs web berbasis PHP yang telah beralih ke versi 8. Sekitar 33,1% masih menggunakan PHP 7.x, dan 8,7% masih berjalan di PHP 5.x [2]. Artinya hampir separuh pengguna PHP di seluruh dunia masih menjalankan versi yang sudah tidak mendapatkan pembaruan keamanan resmi.

Ini bukan masalah kecil. Dukungan resmi untuk PHP 7.4, versi terakhir dalam seri 7.x, berakhir pada 28 November 2022 [3]. Setelah tanggal itu, tidak ada lagi pembaruan keamanan yang dirilis, termasuk untuk kerentanan baru yang terus ditemukan. Basis Data Kerentanan Nasional (NVD) yang dikelola NIST mencatat bahwa sistem yang berjalan di atas versi *end-of-life* secara efektif membiarkan celah keamanan yang sudah diketahui publik tetap terbuka tanpa kemungkinan diperbaiki [4]. Semakin lama sistem dibiarkan berjalan di versi lama, semakin besar akumulasi risiko yang ditanggung.

Jenis kerentanan yang paling umum ditemukan pada aplikasi PHP lama sudah terdokumentasi cukup baik. OWASP Top 10:2021 [5] mencantumkan injeksi sebagai salah satu ancaman paling serius pada aplikasi web. Pada kode PHP lama, injeksi SQL hampir selalu muncul dari penggunaan fungsi `mysql_query()` di mana *input* pengguna langsung digabungkan ke dalam pernyataan SQL tanpa parameterisasi. Kerentanan XSS (*cross-site scripting*) juga lazim karena banyak aplikasi PHP lama tidak menerapkan encode *output* secara konsisten. Selain itu masih ada masalah kriptografi lemah seperti penggunaan `md5()` untuk menyimpan kata sandi, penanganan sesi yang tidak aman, dan berbagai fungsi yang sudah dihapus dari PHP 8.x karena alasan keamanan [5]. Bahwa masalah-masalah ini sudah ada sejak lama bukan

berarti sudah terselesaikan; Merlo, Letarte, dan Antoniol [6] sudah mengangkat persoalan perlindungan otomatis aplikasi PHP dari injeksi SQL sejak 2007, dan solusi komprehensif masih belum tersedia hingga sekarang.

Situasi ini bukan hanya dialami oleh organisasi di luar negeri. FT UGM mengelola sejumlah aplikasi web berbasis PHP yang sebagian di antaranya masih berjalan di atas versi lama, sehingga proses modernisasi menjadi kebutuhan yang mendesak. Kondisi ini merepresentasikan tantangan yang umum dihadapi institusi pendidikan dan pemerintahan yang membangun sistem informasi berbasis PHP pada era sebelum standar pengembangan modern diadopsi secara luas, di mana migrasi sering tertunda karena kompleksitas teknis dan kekhawatiran terhadap gangguan layanan yang sudah berjalan [3].

Secara teknis, migrasi dari PHP 7.x ke PHP 8.x bukan sesuatu yang sederhana. PHP 8.x membawa sejumlah *breaking changes* yang membuat kode PHP 7.x tidak bisa langsung dijalankan tanpa modifikasi [3]. Fungsi-fungsi yang sebelumnya umum digunakan seperti `create_function()`, `each()`, dan seluruh ekstensi `ext/mysql` dihapus. Sistem tipe menjadi jauh lebih ketat dengan tipe *union* dan tipe campuran, yang bisa memicu kesalahan *runtime* pada kode yang sebelumnya berjalan normal. PHP 8.x juga memperkenalkan fitur baru seperti argumen bernama, *match expression*, operator *nullsafe* (`?->`), dan kompilasi JIT [3]. Penanganan kesalahan pun berubah: sejumlah fungsi yang dulu mengembalikan `false` ketika gagal kini melempar pengecualian.

Melakukan semua perubahan ini secara manual pada ratusan atau ribuan *file* kode adalah pendekatan yang tidak realistis. Beberapa organisasi memilih pendekatan bertahap, beralih ke PHP 7.4 dahulu sebelum pindah ke PHP 8.x [3], yang menjadikan dukungan versi target yang dapat dikonfigurasi sebagai kebutuhan nyata, bukan sekadar fitur tambahan.

Alat otomatis yang tersedia saat ini sudah membantu, tapi masing-masing punya batas. Rector [7] adalah alat transformasi berbasis AST yang terbukti andal untuk perubahan sintaksis skala industri, sebagaimana ditunjukkan oleh Chen et al. [8] dalam studi migrasi kode TypeScript di Microsoft. Namun Rector tidak bisa menangani kasus yang membutuhkan pemahaman semantik. Penggantian `mysql_*()` dengan PDO atau MySQLi, misalnya, bukan sekadar mengganti nama fungsi; ini penulisan ulang logika akses basis data secara keseluruhan yang hanya bisa dilakukan jika konteks penggunaannya dipahami.

Di sisi keamanan, Semgrep [9] bisa mengidentifikasi pola kerentanan secara otomatis, dan PHPStan [10] bisa memeriksa kompatibilitas tipe kode dengan versi PHP target. Tapi tidak ada satu pun dari alat-alat ini yang secara inheren menggabungkan

deteksi kerentanan, transformasi kode, dan kepatuhan standar keamanan dalam satu alur kerja yang terintegrasi. Akibatnya tim pengembang yang mengerjakan migrasi dan tim keamanan yang melakukan audit bekerja secara terpisah. Strobl et al. [11] menemukan dari tiga kasus transformasi kode skala besar di sektor pemerintahan bahwa ketiadaan mekanisme validasi yang melekat pada proses transformasi menimbulkan risiko yang baru terasa jauh setelah sistem berjalan di produksi.

Tekanan bertambah bagi organisasi yang beroperasi di bawah standar keamanan informasi. ISO/IEC 27001:2022 [12] mensyaratkan praktik *secure coding* yang terdokumentasi dan dapat diaudit melalui tiga kontrol dalam Annex A.8: A.8.25 (*secure development lifecycle*), A.8.28 (*secure coding*), dan A.8.29 (pengujian keamanan dalam pengembangan dan penerimaan). Bagi organisasi yang sedang dalam proses sertifikasi ISO/IEC 27001:2022, bukti bahwa migrasi kode dilakukan dengan memperhatikan standar ini harus tersedia sebagai bagian dari dokumentasi audit. Menghasilkan bukti tersebut saat ini masih memerlukan pekerjaan manual yang terpisah dari proses migrasi.

Kemajuan dalam *Large Language Models* (LLM) membuka kemungkinan baru. LLM sudah terbukti dapat digunakan untuk mendeteksi kerentanan perangkat lunak [13] [14]. Li, Dutta, dan Naik [15] menunjukkan bahwa menggabungkan LLM dengan alat analisis statis menghasilkan deteksi kerentanan yang lebih baik dari masing-masing pendekatan yang berjalan sendiri. Stümpfle et al. [16] meneliti penggunaan LLM untuk transformasi kode yang membutuhkan pemahaman semantik dalam konteks migrasi varian perangkat lunak. Dengan kemampuan memahami konteks kode, LLM berpotensi mengisi celah yang tidak bisa ditangani alat berbasis aturan seperti Rector.

Kendala utamanya adalah privasi data. Model LLM berperforma tinggi seperti GPT-4 diakses melalui API *cloud*, yang berarti kode sumber harus dikirimkan ke *server* pihak ketiga. Untuk institusi seperti FT UGM yang mengelola kode sumber aplikasi internal yang bersifat rahasia, ini bukan pilihan yang bisa diterima. Perkembangan model LLM *open source* berukuran kecil hingga menengah beberapa tahun terakhir mengubah situasi ini. Model-model seperti DeepSeek Coder 6.7b [17], Qwen2.5-Coder 7b [18], CodeLlama 7b [19], Mistral 7b [20], dan Llama 3.1 8b [21] kini bisa dijalankan secara lokal pada perangkat keras konsumen ber-VRAM 8 GB melalui Ollama [22]. Seluruh proses inferensi terjadi di mesin lokal tanpa ada data yang keluar dari lingkungan organisasi.

Dari uraian di atas, ada dua masalah mendasar yang belum terpecahkan sekaligus. Pertama, migrasi kode PHP lama masih memerlukan intervensi manual besar untuk kasus yang butuh pemahaman semantik, dan alat yang ada tidak menyertakan pemeriksaan keamanan sebagai bagian dari alur migrasi. Kedua, organisasi yang harus memenuhi ISO/IEC 27001:2022 masih harus melakukan audit keamanan sebagai langkah terpisah

setelah migrasi selesai. Belum ada platform yang menggabungkan keduanya dalam satu alur kerja *open source* yang dapat dijalankan secara lokal, sebagaimana ditunjukkan oleh tinjauan literatur pada Bab II.

Penelitian ini membangun *platform* berbasis AI dan alat *open source* yang dapat secara otomatis mengubah aplikasi web PHP lama ke versi PHP target yang dapat dikonfigurasi, dengan dukungan teknis dari PHP 5.2 hingga PHP 8.x, sekaligus menyertakan pemeriksaan *secure coding* berbasis ISO/IEC 27001:2022. *Platform* ini menggunakan Rector [7] untuk transformasi kode berbasis AST, Semgrep [9] untuk pemindaian keamanan, PHPStan [10] untuk validasi pasca-konversi, serta lima model LLM *open source* yang dijalankan secara lokal melalui Ollama [22]. Studi kasus dilakukan pada kode PHP milik FT UGM.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, penelitian ini merumuskan tiga permasalahan sebagai berikut:

1. Bagaimana merancang sistem berbasis AI yang mampu melakukan konversi otomatis aplikasi web PHP lama ke versi PHP tertentu sambil mempertahankan fitur aplikasi?
2. Bagaimana mengintegrasikan pemeriksaan *secure coding* yang selaras dengan standar ISO/IEC 27001:2022 ke dalam proses konversi otomatis?
3. Bagaimana memvalidasi bahwa hasil konversi memenuhi persyaratan fungsional dan keamanan ISO/IEC 27001:2022?

1.3 Tujuan Penelitian

Penelitian ini memiliki empat tujuan yang menjawab rumusan masalah di atas:

1. Mengembangkan *platform* berbasis AI yang menggunakan alat *open source* untuk secara otomatis mengubah aplikasi web PHP lama ke versi PHP target yang dapat dikonfigurasi tanpa kehilangan fungsionalitasnya.
2. Mengimplementasikan mekanisme pemeriksaan *secure coding* yang terhubung dengan kontrol dalam ISO/IEC 27001:2022 Annex A.8 [12], yang akan menjaga keamanan kode yang dikonversi.
3. Membangun sistem *quality assurance* yang memeriksa hasil konversi terhadap kebenaran fungsional dan standar kepatuhan keamanan yang ditetapkan oleh ISO/IEC 27001:2022 [12].
4. Menghasilkan dokumentasi otomatis berupa laporan konversi dan pemetaan kepatuhan terhadap ISO/IEC 27001:2022 [12], untuk membantu audit keamanan

informasi.

1.4 Batasan Penelitian

Penelitian ini memiliki batasan-batasan sebagai berikut:

1. **Ruang lingkup konversi:** Secara teknis, *pipeline* yang dibangun mendukung konversi dari PHP 5.2 ke atas, karena Rector menggunakan rantai *ruleset* kumulatif yang mencakup semua perubahan dari PHP 5.2 hingga versi target yang dipilih [7]. Namun studi kasus dalam penelitian ini menggunakan kode sumber PHP 7.x milik FT UGM sebagai representasi kondisi yang paling umum ditemukan saat ini [2]. Versi target yang dapat dikonfigurasi mencakup PHP 7.4 hingga PHP 8.3, yang merupakan rentang versi paling relevan untuk kebutuhan modernisasi institusi saat ini [3].
2. **Studi kasus:** Fokus utama studi kasus ini adalah aplikasi *Dashboard TCK (Target Capaian Kerja)* milik Fakultas Teknik (FT) Universitas Gadjah Mada, yang dibangun di atas *framework* CodeIgniter 3 dengan versi PHP 7.4.9. *Pipeline* yang dikembangkan bekerja pada kode *backend* PHP di direktori *application/* dan tidak menyentuh komponen *frontend* Vue 2 SPA. *Pipeline* bekerja pada level *file* PHP individual dan tidak bergantung pada *framework* yang digunakan, sehingga secara teknis dapat diterapkan pada aplikasi berbasis *framework* lain seperti Laravel atau Symfony maupun pada aplikasi PHP tanpa *framework*. Pengujian terhadap *framework* selain CodeIgniter 3 bukan bagian dari ruang lingkup penelitian ini.
3. **Kontrol keamanan:** Pemeriksaan keamanan hanya dilakukan pada kontrol ISO/IEC 27001:2022 Annex A yang relevan dengan *secure coding*: A.8.25, A.8.28, dan A.8.29, serta A.5.17 dan A.8.24 dan A.8.26 yang turut dipetakan oleh *pipeline* [12].
4. **Model AI:** Penelitian ini mengevaluasi dan membandingkan lima model LLM *open source* yang dijalankan secara lokal melalui Ollama [22] dalam eksperimen komparasi *zero-shot*: DeepSeek Coder 6.7b [17], Qwen2.5-Coder 7b [18], CodeLlama 7b [19], Mistral 7b [20], dan Llama 3.1 8b [21]. Semua model digunakan tanpa *fine-tuning*.
5. **Lingkungan validasi:** Pengujian dilakukan dalam pengaturan lokal menggunakan kode sumber dari lingkungan institusional. Pengujian dalam lingkungan produksi skala besar bukan bagian dari ruang lingkup penelitian ini.
6. **Bahasa pemrograman:** Penelitian ini terbatas pada aplikasi PHP. Bahasa lain tidak dicakup.
7. **Kompleksitas aplikasi:** Aplikasi uji memiliki kompleksitas tingkat menengah,

dengan maksimum 10.000 baris kode.

1.5 Manfaat Penelitian

1.5.1 Manfaat Akademis

Penelitian ini memberikan contoh nyata tentang cara mengintegrasikan kontrol *secure coding* ISO/IEC 27001:2022 ke dalam alur kerja transformasi kode otomatis [12], sesuatu yang belum dilakukan oleh penelitian-penelitian sebelumnya di bidang ini [6] [13] [16]. Selain itu, penelitian ini menghasilkan data empiris tentang performa model-model LLM *open source* lokal berukuran kecil hingga menengah dalam tugas analisis keamanan kode, yang relevan bagi institusi dengan sumber daya komputasi terbatas.

Desain modular *platform* memungkinkan pengembangan lebih lanjut oleh peneliti lain, misalnya dengan mengganti alat transformasi untuk mendukung bahasa pemrograman lain, menggunakan model AI yang lebih baru, atau memperluas pemetaan ke standar keamanan lain selain ISO/IEC 27001:2022.

1.5.2 Manfaat Praktis

Platform yang dibangun memudahkan pemindahan aplikasi PHP lama, khususnya bagi organisasi yang tidak memiliki banyak sumber daya pengembangan. Pemeriksaan kepatuhan keamanan disertakan langsung dalam proses migrasi [12], sehingga kerentanan dapat ditemukan bersamaan dengan konversi dan laporan kepatuhan dihasilkan secara otomatis. Dukungan versi target dari PHP 5.2 hingga PHP 8.x mengakomodasi berbagai strategi migrasi, baik yang bertahap maupun yang langsung. Seluruh *platform* bersifat *open source* dan dapat digunakan secara gratis.

1.6 Sistematika Penulisan

Skripsi ini terdiri dari lima bab.

Bab I: Pendahuluan memaparkan latar belakang masalah, rumusan masalah, tujuan penelitian, batasan penelitian, manfaat penelitian, dan sistematika penulisan.

Bab II: Tinjauan Pustaka dan Landasan Teoritis membahas penelitian-penelitian sebelumnya yang relevan dan kerangka teoritis, mencakup perubahan versi PHP, standar ISO/IEC 27001:2022, metodologi *secure coding*, analisis kode berbasis LLM, serta alat-alat yang digunakan: Rector, Semgrep, PHPStan, dan Ollama.

Bab III: Metodologi Penelitian menjelaskan arsitektur sistem, tahapan penelitian, desain *prompt* LLM, pemetaan kontrol ISO/IEC 27001:2022, rancangan eksperimen komparasi model, serta metode pengujian dan evaluasi.

Bab IV: Hasil dan Diskusi menyajikan hasil implementasi pada studi kasus website PHP FT UGM, mencakup tingkat keberhasilan konversi, perbandingan temuan keamanan sebelum dan sesudah konversi, hasil komparasi lima model LLM, dan diskusi mengenai kekuatan dan keterbatasan sistem.

Bab V: Kesimpulan dan Rekomendasi merangkum temuan berdasarkan tujuan yang ditetapkan dan memberikan saran untuk penelitian berikutnya.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Literatur

Bagian ini membahas penelitian-penelitian sebelumnya yang relevan dengan tiga fokus utama studi ini: analisis keamanan otomatis pada aplikasi PHP, deteksi kerentanan berbasis LLM, dan transformasi kode yang melibatkan LLM. Masing-masing karya dibahas dari sisi masalah yang diatasi, kontribusinya, dan keterbatasannya. Bagian ini ditutup dengan tabel perbandingan yang memperlihatkan posisi penelitian ini di antara karya-karya tersebut.

2.1.1 Perlindungan Otomatis Aplikasi PHP dari Injeksi SQL

Merlo, Letarte, dan Antoniol [6] pada 2007 sudah menunjukkan bahwa analisis keamanan dan perbaikan kode PHP dapat diotomatisasi. Pendekatan mereka menggabungkan analisis statis, analisis dinamis, dan rekayasa ulang kode untuk melindungi aplikasi PHP lama dari injeksi SQL tanpa mengubah fungsionalitasnya. Caranya adalah dengan mengekstrak penjaga berbasis model dari kasus uji tepercaya, lalu secara otomatis menyisipkannya ke titik-titik akses basis data dalam kode sumber.

Hasilnya cukup kuat: pengujian pada phpBB versi 2.0.0 (37.193 baris kode PHP 4.2.2) berhasil menghentikan semua 176 serangan injeksi yang diuji tanpa satu pun *false positive*. Tapi ada batasnya. Kualitas perlindungan sepenuhnya bergantung pada kelengkapan rangkaian pengujian yang digunakan selama fase analisis dinamis. Titik akses yang tidak tercakup pengujian tidak terlindungi.

Yang relevan dari karya ini untuk penelitian saat ini bukan teknisnya, tapi konfirmasinya: otomatisasi analisis keamanan dan remediasi kode PHP dalam skala besar memang bisa dilakukan. Fakta bahwa studi ini sudah muncul hampir dua dekade lalu, namun solusi komprehensif yang bekerja dengan standar keamanan modern masih belum ada, justru mempertegas kesenjangan yang ingin diisi oleh penelitian ini.

2.1.2 Menggunakan LLM untuk Menemukan dan Memantau Kerentanan

Akuthota dkk. [13] meneliti kemampuan GPT-3.5-Turbo untuk deteksi dan pemantauan kerentanan perangkat lunak. Pendekatannya murni berbasis *prompt engineering* tanpa pelatihan ulang model sama sekali. Model berhasil menemukan pola kerentanan yang umum, meskipun akurasi menurun seiring meningkatnya kompleksitas pola yang dicari.

Li et al. [14] mengambil pendekatan yang lebih terstruktur. Mereka menggabungkan analisis *patch* historis dengan analisis statis, lalu menambahkan

tabel fitur kerentanan dan templat penganalisis untuk membuat klasifikasi LLM lebih akurat, terutama pada kerentanan yang membutuhkan pemahaman aliran data lintas fungsi. Evaluasi pada *dataset* CVE kernel Linux memperlihatkan hasil yang lebih baik dibandingkan analisis statis saja.

Dua temuan penting bisa ditarik dari kedua studi ini. LLM tujuan umum sudah bisa menemukan pola kerentanan hanya dengan *prompt engineering*, dan penggabungan LLM dengan analisis statis terstruktur memberikan hasil yang lebih akurat untuk kasus yang kompleks. Hambatan yang sama muncul di keduanya: model dijalankan via API *cloud*, yang menjadi kendala nyata ketika kode yang dianalisis bersifat rahasia. Penelitian ini mengatasi kendala tersebut dengan menjalankan semua model secara lokal melalui Ollama, sehingga tidak ada kode sumber yang keluar dari lingkungan organisasi.

2.1.3 Transformasi Kode dengan Bantuan LLM

Stümpfle dkk. [16] mengusulkan pendekatan untuk mengubah varian perangkat lunak yang dikloning menjadi *Software Product Line* (SPL) menggunakan LLM. Metodenya bekerja dalam tiga tahap: penyaringan titik variasi untuk menemukan perbedaan kode yang relevan secara semantik antar varian, rekayasa *prompt* untuk menghasilkan artefak SPL yang benar, dan *loop* umpan balik penyempurnaan diri yang mengirimkan hasil pengujian kembali ke model secara berulang hingga kode lolos kompilasi dan pengujian fungsional. *Loop* umpan balik ini terbukti secara signifikan mengurangi kegagalan akibat halusinasi model, meskipun efektivitasnya tetap bergantung pada kualitas langkah penyaringan awal.

Relevansinya dengan penelitian ini cukup langsung. Keduanya menggunakan LLM untuk transformasi kode yang melampaui substitusi sintaksis dan membutuhkan pemahaman semantik. *Loop* umpan balik dalam Stümpfle dkk. memiliki fungsi konseptual yang mirip dengan peran validasi PHPStan dalam *pipeline* penelitian ini: keduanya menggunakan *output* alat analisis statis untuk memverifikasi apakah transformasi berhasil. Perbedaananya terletak pada fokus: Stümpfle et al. menangani ekstraksi fitur lintas varian untuk rekayasa lini produk, sedangkan penelitian ini berfokus pada migrasi versi dan verifikasi kepatuhan keamanan untuk kode warisan dalam satu bahasa.

2.1.4 Pendekatan Neuro-Simbolik untuk Deteksi Kerentanan Keamanan

Li, Dutta, dan Naik [15] memperkenalkan IRIS, sistem yang menggabungkan LLM dengan analisis statis untuk deteksi kerentanan pada tingkat seluruh repositori. Masalah yang diangkat cukup spesifik: alat analisis statis tradisional seperti CodeQL sangat bergantung pada spesifikasi *taint* yang dibuat manual, dan spesifikasi ini hampir pasti tidak lengkap karena jumlah API pihak ketiga terus bertambah. LLM juga tidak bisa

berjalan sendiri karena tidak mampu melakukan penalaran yang mencakup keseluruhan aliran kode dalam proyek berskala besar.

Solusi IRIS adalah membiarkan LLM menghasilkan spesifikasi *taint* secara otomatis berdasarkan pemahaman kontekstual terhadap API yang relevan, lalu menyerahkan analisis jalur ke alat analisis statis yang sudah ada. Pada *dataset* CWE-Bench-Java yang terdiri dari 120 kerentanan yang divalidasi secara manual, IRIS dengan GPT-4 berhasil mendeteksi 55 kerentanan dibandingkan hanya 27 oleh CodeQL saja, sekaligus menurunkan rata-rata *false positive* sebesar 5,21 poin persentase.

Hasil ini menjadi dasar empiris pemilihan pendekatan gabungan dalam penelitian ini. Bedanya: IRIS mengandalkan GPT-4 via *cloud* dan hanya diuji pada Java. Penelitian ini mengambil pendekatan konseptual yang serupa, tetapi dengan model lokal melalui Ollama dan menargetkan PHP.

2.1.5 Dampak Jangka Panjang Transformasi Kode Otomatis

Strobl dkk. [11] menganalisis tiga kasus transformasi kode otomatis skala besar di sebuah organisasi pemerintah Austria, semuanya melibatkan konversi sistem warisan dari COBOL dan PL/I ke Java dengan ukuran lebih dari satu juta baris kode. Yang membedakan studi ini dari yang lain adalah fokusnya bukan pada keberhasilan transformasi itu sendiri, melainkan pada apa yang terjadi setelahnya.

Hasilnya mengganggu. Transformasi otomatis seringkali menghasilkan kode yang sintaksis valid di bahasa target, tapi mempertahankan karakteristik struktural dari bahasa sumber sehingga sulit dipahami pengembang yang tidak kenal sistem lama. Lebih kritis lagi: kasus yang memiliki *test suite* otomatis sejak awal jauh lebih mudah dirawat pascatransformasi. Kasus tanpa pengujian otomatis menjadi beban jangka panjang.

Temuan ini secara langsung memperkuat keputusan desain dalam penelitian ini untuk menyertakan PHPStan sebagai validasi pasca-konversi dan Semgrep sebagai pemindaian keamanan sebelum dan sesudah transformasi. Masalah yang ditinggalkan oleh konversi otomatis perlu terdeteksi segera, bukan bertahun-tahun kemudian.

2.1.6 Transformasi Kode Berbasis Pola untuk Migrasi Aplikasi

Cai et al. [23] mengusulkan pendekatan transformasi kode berbasis pola untuk migrasi aplikasi ke lingkungan *cloud*. Prosesnya iteratif: pemindaian kode untuk mengidentifikasi blok yang perlu diubah, pencocokan pola menggunakan ekspresi reguler yang diperluas, perancangan templat untuk kode target, lalu eksekusi aturan transformasi. Pendekatan ini diuji pada 19 proyek *open source* Java dengan memigrasikannya ke Amazon Web Services (AWS).

Kontribusinya jelas: transformasi berbasis pola terbukti bisa mengurangi

pekerjaan manual berulang pada migrasi skala besar. Tapi ada keterbatasan mendasar yang tidak bisa dihindari. Ekspresi reguler yang diperluas tetap beroperasi pada level teks, bukan pada representasi struktural kode. Ini membuatnya rentan gagal pada konstruksi kode yang kompleks atau tidak mengikuti pola yang telah didefinisikan sebelumnya.

Rector [7] bekerja pada AST, bukan teks, sehingga transformasinya lebih dapat diandalkan untuk kode produksi yang beragam. Selain itu, Cai et al. [23] tidak menyertakan aspek keamanan atau kepatuhan standar sama sekali dalam pendekatan mereka. Perbedaan ini mencerminkan bagaimana kebutuhan dalam transformasi kode otomatis telah berkembang: dari sekadar memindahkan kode ke lingkungan baru, menjadi memastikan kode yang dipindahkan juga aman.

2.1.7 Migrasi Kode Berbasis AST pada Skala Industri

Chen et al. [8] mendokumentasikan penerapan migrasi kode berbasis AST pada skala produksi di Microsoft. Pendekatan mereka menggunakan penggantian simbol berbasis AST untuk memigrasikan basis kode TypeScript berskala besar secara otomatis. Kunci dari pendekatan ini adalah bahwa transformasi dilakukan pada tingkat pohon sintaksis, bukan teks, sehingga menghasilkan perubahan yang struktural konsisten dan tidak bergantung pada gaya penulisan kode masing-masing pengembang.

Studi ini penting bukan karena inovasinya yang radikal, tapi karena validasi skalanya. Dengan membuktikan bahwa transformasi berbasis AST bisa dijalankan pada jutaan baris kode produksi tanpa regresi fungsional yang signifikan, Chen et al. mengkonfirmasi bahwa pilihan teknis yang sama yang mendasari Rector layak diandalkan dalam konteks institusional nyata.

Ada dua perbedaan konteks yang perlu dicatat. Pertama, migrasi yang dilakukan Chen et al. adalah TypeScript ke TypeScript (*refactoring* dalam bahasa yang sama), sedangkan penelitian ini berhadapan dengan *breaking changes* antar versi PHP yang melibatkan penghapusan API dan perubahan sistem tipe. Kedua, seperti halnya Cai et al. [23], studi ini tidak menyentuh aspek keamanan sama sekali. Temuan keamanan yang muncul selama atau setelah migrasi bukan bagian dari cakupan mereka.

2.1.8 Ringkasan Perbandingan

Tabel 2.1. Perbandingan Penelitian Terdahulu dan Karakteristik Penelitian

Studi	Bahasa	AI/LLM Digunakan	Tugas Utama
Merlo et al. [6]	PHP	Tidak ada	<i>Security remediation</i>
Akuthota et al. [13]	<i>Multiple</i>	GPT-3.5-Turbo	<i>Vulnerability detection</i>
Li et al. [14]	C (Linux kernel)	LLM + analisis statis	<i>Vulnerability detection</i>
Stümpfle et al. [16]	C/C++	LLM	<i>Code transformation</i>
Li, Dutta & Naik [15]	Java	GPT-4	<i>Vulnerability detection</i>
Strobl et al. [11]	COBOL/PL/I	Tidak ada	<i>Code transformation</i>
Cai et al. [23]	Java	Tidak ada	<i>Cloud migration</i>
Chen et al. [8]	TypeScript	Tidak ada	<i>Code migration (AST)</i>
Penelitian ini	PHP	5 model LLM lokal	Migrasi versi + <i>security compliance</i>

Tabel 2.2. Perbandingan Penelitian Terdahulu dan Aspek Metodologi

Studi	<i>Deployment</i>	Validasi Pasca-Konversi	Standar Keamanan	<i>Open Source</i>
Merlo et al. [6]	N/A	Tidak	Tidak ada	Ya
Akuthota et al. [13]	<i>Cloud</i>	Tidak	Tidak ada	Tidak
Li et al. [14]	<i>Cloud</i>	Tidak	Tidak ada	Tidak
Stümpfle et al. [16]	<i>Cloud</i>	Ya (<i>feedback loop</i>)	Tidak ada	Tidak
Li, Dutta & Naik [15]	<i>Cloud</i>	Tidak	Tidak ada	Ya (IRIS)
Strobl et al. [11]	N/A	Tidak	Tidak ada	Tidak
Cai et al. [23]	N/A	Tidak	Tidak ada	Tidak
Chen et al. [8]	N/A	Tidak	Tidak ada	Tidak
Penelitian ini	Lokal (Ollama)	Ya (PHPStan + Semgrep)	ISO/IEC 27001:2022	Ya

Dari kedua tabel terlihat kesenjangan yang konsisten. Studi-studi yang berfokus pada keamanan tidak menangani migrasi versi. Studi-studi yang berfokus pada transformasi kode, termasuk yang sudah terbukti di skala industri seperti Chen et al. [8] tidak menyertakan pemeriksaan keamanan maupun pemetaan ke standar formal. Tidak satu pun dari kedelapan studi yang menggabungkan migrasi versi PHP, analisis keamanan berbasis LLM, validasi pasca-konversi, dan kepatuhan ISO/IEC 27001:2022 dalam satu alur kerja *open source* dengan *deployment* lokal. Kombinasi inilah yang menjadi kontribusi utama penelitian ini.

2.2 Dasar Teori

2.2.1 PHP dan Perubahan Antar Versi

PHP (*Hypertext Preprocessor*) menjadi bahasa skrip sisi *server* yang paling banyak digunakan di web sejak akhir 1990-an [1]. W3Techs mencatat bahwa 71,7% dari seluruh situs web yang memiliki bahasa pemrograman sisi *server* menggunakan PHP [2].

Untuk memahami tantangan migrasi yang dihadapi kode lama, perlu dipahami dulu karakteristik PHP 5.x yang masih banyak ditemukan di sistem yang belum bermigrasi. PHP 5.x, aktif digunakan dari sekitar 2004 hingga awal 2010-an, memiliki karakteristik yang berbeda secara fundamental dari versi modern. Yang paling signifikan adalah ekstensi `ext/mysql` dengan fungsi-fungsi seperti `mysql_connect()`, `mysql_query()`, dan `mysql_fetch_array()`. Fungsi-fungsi ini tidak mendukung *prepared statements* secara asli, sehingga rentan terhadap injeksi SQL jika *input* pengguna tidak disanitasi secara manual. Ekstensi ini sudah dinyatakan *deprecated* sejak PHP 5.5 dan dihapus sepenuhnya di PHP 7.0 [3]. PHP 5.x juga menggunakan kata kunci `var` untuk deklarasi properti kelas, sintaks `array()` yang panjang, dan belum mendukung *type hints* skalar. Rector dapat menangani sebagian besar transformasi sintaksis dari PHP 5.x melalui *ruleset* kumulatifnya [7], tapi penggantian `mysql_*()` tidak bisa dilakukan Rector karena butuh pemahaman semantik tentang konteks penggunaan basis data. Keterbatasan inilah yang secara langsung memotivasi kehadiran AI Engine dalam *pipeline* penelitian ini.

PHP 7.x membawa peningkatan performa signifikan lewat Zend Engine 3.0, menambahkan deklarasi tipe skalar, petunjuk tipe pengembalian, dan operator penggabungan *null*. PHP 7.4, versi terakhir seri ini, menambahkan properti bertipe dan fungsi panah sebelum dukungan resminya berakhir 28 November 2022 [3].

PHP 8.x membuat sejumlah *breaking changes* yang membuat kode PHP 7.x tidak bisa langsung dijalankan [3]. Selain penghapusan fungsi-fungsi yang sudah *deprecated*, sistem tipe menjadi jauh lebih ketat dengan tipe *union* dan tipe campuran. Fitur baru seperti *match expression*, operator *nullsafe* (`?->`), argumen bernama, dan

kompilasi JIT diperkenalkan. Penanganan kesalahan juga berubah: beberapa fungsi yang sebelumnya mengembalikan `false` kini melempar pengecualian. Untuk basis kode besar, melakukan semua perubahan ini secara manual jelas tidak realistis.

2.2.2 ISO/IEC 27001:2022 dan *Secure Coding*

ISO/IEC 27001:2022 adalah standar internasional untuk Sistem Manajemen Keamanan Informasi (ISMS) yang dikembangkan bersama oleh ISO dan IEC [12]. Standar ini menetapkan persyaratan untuk membuat, menerapkan, memelihara, dan terus meningkatkan ISMS dalam suatu organisasi. Annex A memuat 93 kontrol keamanan yang dibagi ke dalam empat domain: Organisasi, Manusia, Fisik, dan Teknologi. Dari 93 kontrol tersebut, penelitian ini hanya memetakan yang langsung relevan dengan pengembangan perangkat lunak:

- **A.8.25** (*Secure development lifecycle*) mengharuskan aturan pengembangan yang aman ditetapkan dan diikuti di setiap tahap, termasuk penggunaan pedoman *secure coding* dan alat analisis statis.
- **A.8.28** (*Secure coding*) mengharuskan aturan pengkodean yang aman diikuti untuk mencegah kerentanan umum seperti injeksi, autentikasi yang rusak, pengungkapan data sensitif, dan kriptografi yang tidak aman.
- **A.8.29** (Pengujian keamanan dalam pengembangan dan penerimaan) mengharuskan pengujian keamanan menjadi bagian dari proses pengembangan sebelum kode masuk produksi.

Ketiga kontrol ini, bersama A.5.17, A.8.24, dan A.8.26 yang turut dipetakan oleh *pipeline*, menjadi acuan resmi untuk mesin pengecekan keamanan dalam penelitian ini [12]. Aturan Semgrep yang digunakan berhubungan langsung dengan A.8.28 dan A.8.29. Struktur *pipeline* secara keseluruhan mencerminkan kebutuhan siklus hidup A.8.25.

2.2.3 OWASP Top 10 dan Kerentanan Aplikasi Web PHP

OWASP Top 10 adalah referensi standar untuk risiko keamanan paling kritis pada aplikasi web yang diterbitkan secara berkala oleh OWASP berdasarkan data dari ratusan organisasi di seluruh dunia [5]. Versi terbaru, OWASP Top 10:2021, relevan langsung dengan kondisi aplikasi PHP lama.

Injeksi menempati posisi ketiga dan merupakan temuan paling umum pada kode PHP lama [5]. Injeksi SQL sering muncul dari penggunaan `mysql_query()` tanpa parameterisasi. XSS juga termasuk kategori injeksi dan terjadi saat *output* tidak diekode sebelum ditampilkan ke peramban. Kegagalan kriptografi, yang berada di posisi kedua, kerap muncul dalam praktik penyimpanan kata sandi menggunakan `md5()` atau `sha1()`, padahal keduanya sudah lama tidak dianggap aman.

Dalam penelitian ini, Semgrep dijalankan dengan *ruleset* `p/php` dan `p/owasp-top-ten` [9], yang secara langsung dipetakan ke kategori-kategori dalam OWASP Top 10:2021. Temuan Semgrep kemudian dipetakan ke kontrol ISO/IEC 27001:2022 yang relevan melalui skrip `iso_mapper.py`.

2.2.4 DevSecOps dan Integrasi Keamanan dalam *Pipeline* Pengembangan

DevSecOps adalah praktik yang mengintegrasikan keamanan langsung ke dalam setiap tahap siklus hidup pengembangan perangkat lunak [14]. Berbeda dari pendekatan tradisional yang menempatkan pengujian keamanan sebagai langkah terpisah di akhir proses, DevSecOps memperlakukan keamanan sebagai bagian dari setiap keputusan teknis selama pengembangan berlangsung, bukan tanggung jawab tim khusus yang bekerja setelah kode selesai ditulis.

Dalam praktiknya, DevSecOps diwujudkan melalui integrasi alat keamanan otomatis ke dalam *pipeline*: analisis statis kode sumber (SAST), pemindaian dependensi, dan pengujian keamanan dinamis. Kerentanan yang ditemukan lebih awal jauh lebih murah untuk diperbaiki dibandingkan yang baru ditemukan setelah sistem berjalan di produksi.

Pipeline dalam penelitian ini menerapkan prinsip DevSecOps secara langsung: Semgrep memindai sebelum dan sesudah konversi, PHPStan memvalidasi kompatibilitas kode, dan AI Engine menangani kasus yang butuh analisis semantik. Keamanan bukan langkah audit terpisah, tapi bagian dari proses migrasi itu sendiri [12].

2.2.5 Abstract Syntax Trees dan Static Analysis

Analisis statis adalah teknik pemeriksaan kode sumber tanpa menjalankan program. Dalam konteks keamanan perangkat lunak, yang digunakan untuk mengidentifikasi pola kerentanan, praktik pengkodean tidak aman, dan pelanggaran kebijakan secara otomatis. Analisis statis sudah menjadi bagian standar dari *pipeline* DevSecOps modern karena bisa dijalankan terus-menerus selama pengembangan tanpa beban tambahan yang signifikan [14].

Abstract Syntax Trees (AST) adalah representasi pohon dari kode sumber di mana setiap *node* mewakili konstruksi sintaksis seperti ekspresi, pernyataan, atau deklarasi. Alat yang bekerja dengan AST bisa menganalisis dan mengubah kode secara struktural, berbeda dari pendekatan berbasis teks biasa yang digunakan oleh Cai et al. [23]. Rector menggunakan PHP AST untuk menemukan pola yang perlu diubah dan menghasilkan kode penggantinya. Chen et al. [8] mendemonstrasikan bahwa pendekatan berbasis AST bukan sekadar teori; mereka menerapkannya untuk migrasi kode TypeScript di Microsoft pada skala produksi.

2.2.6 Rector: Alat Transformasi Kode Berbasis AST

Rector adalah alat *open source* untuk transformasi kode PHP berbasis AST [7]. Prosesnya: kode sumber PHP diubah menjadi AST, aturan transformasi diterapkan, lalu kode PHP baru dihasilkan dari pohon yang sudah dimodifikasi. Rector dilengkapi aturan bawaan untuk berpindah antar versi PHP, termasuk `LevelSetList::UP_TO_PHP_80` hingga `UP_TO_PHP_83`, dan *ruleset*-nya bersifat kumulatif sehingga mencakup semua perubahan dari versi sumber hingga versi target.

Dalam penelitian ini, Rector berperan sebagai mesin transformasi utama. Transformasi sintaksis dan struktural dari PHP 5.x atau 7.x ke PHP 8.x ditangani sepenuhnya oleh Rector. Yang tidak bisa ditangani adalah penggantian `ext/mysql` karena itu butuh penulisan ulang semantik, bukan sekadar penggantian sintaks. Itulah yang ditangani oleh AI Engine.

2.2.7 PHPStan: Alat Analisis Statis untuk PHP

PHPStan adalah alat analisis statis untuk PHP yang memeriksa kebenaran tipe data, konsistensi pemanggilan fungsi, dan kesalahan logika tanpa menjalankan program [10]. Berbeda dari Semgrep yang mencari pola kerentanan keamanan, PHPStan fokus pada integritas kode dari sisi kompatibilitas dan tipe.

PHPStan bekerja dengan sistem level 0–9, di mana level lebih tinggi berarti pemeriksaan lebih ketat. Dalam *pipeline* penelitian ini, PHPStan digunakan pada level 5 sebagai komponen validasi pasca-konversi. Setelah Rector selesai, PHPStan memeriksa apakah kode hasil konversi kompatibel dengan versi PHP target dan bebas dari kesalahan tipe. Perannya mendukung kontrol A.8.29 ISO/IEC 27001:2022 [12] dengan memastikan kode yang dihasilkan secara otomatis tidak mengandung kesalahan yang bisa memengaruhi keandalan sebelum masuk produksi.

2.2.8 Semgrep: Alat Analisis Statis Ringan untuk Keamanan

Semgrep adalah alat analisis statis ringan yang mendukung lebih dari 30 bahasa pemrograman, termasuk PHP [9]. Aturan keamanannya ditulis dalam format YAML dengan sintaks yang mirip dengan kode sumber yang dianalisis, sehingga pengguna bisa membuat aturan sendiri tanpa perlu memahami detail AST bahasa target. Registri aturan publiknya mencakup berbagai kerentanan umum seperti injeksi SQL, XSS, kriptografi lemah, dan deserialisasi tidak aman.

Dalam penelitian ini, Semgrep digunakan dua kali: sebelum konversi untuk menetapkan *baseline* keamanan kode asli, dan sesudah konversi untuk memastikan tidak ada kerentanan baru yang muncul. Kedua pemindaian ini mendukung kontrol A.8.28

dan A.8.29 ISO/IEC 27001:2022 [12].

2.2.9 Model-Model LLM untuk Analisis Kode

LLM (*Large Language Model*) adalah model bahasa neural berbasis *transformer* yang dilatih pada data teks dan kode dalam jumlah besar. LLM yang dilatih pada kode bisa memahami semantik kode, mengidentifikasi pola kerentanan, menyarankan perbaikan, dan menghasilkan kode baru [13]. Kemampuan ini berbeda dari analisis berbasis aturan: LLM mempertimbangkan konteks dan tujuan kode, bukan hanya strukturnya.

Penelitian ini mengevaluasi lima model LLM *open source* yang dijalankan secara lokal. Kriteria pemilihan ada dua: model harus bisa dijalankan di GPU dengan VRAM 8 GB menggunakan kuantisasi Q4, dan harus memiliki kemampuan pemahaman kode yang terdokumentasi dalam literatur ilmiah.

DeepSeek Coder 6.7b [17] memiliki 6,7 miliar parameter dan dilatih pada 2 triliun token dengan komposisi 87% kode sumber dari lebih dari 80 bahasa pemrograman.

Qwen2.5-Coder 7b [18] adalah model kode dari Alibaba dengan 7 miliar parameter, dilatih pada 5,5 triliun token kode, dan menunjukkan kemampuan yang kuat pada penyelesaian kode dan perbaikan *bug*.

CodeLlama 7b [19] dikembangkan Meta AI berbasis Llama 2. Model ini mendukung konteks hingga 100 ribu token dan tersedia dalam varian *instruct* yang dioptimalkan untuk mengikuti instruksi.

Mistral 7b [20] adalah model tujuan umum dari Mistral AI yang menggunakan mekanisme *grouped-query attention* (GQA) dan *sliding window attention* (SWA), membuatnya efisien secara komputasi.

Llama 3.1 8b [21] adalah model 8 miliar parameter dari Meta AI yang dilatih pada lebih dari 15 triliun token multibahasa dengan peningkatan signifikan dalam kemampuan penalaran dibanding pendahulunya.

Li, Dutta, dan Naik [15] sudah menunjukkan bahwa kombinasi LLM dan analisis statis menghasilkan deteksi kerentanan yang lebih baik dari masing-masing yang berjalan sendiri. Perbandingan performa kelima model di atas dalam konteks analisis keamanan kode PHP akan dibahas secara empiris di Bab IV.

2.2.10 Ollama: Menerapkan LLM Secara Lokal

Ollama adalah *platform open source* untuk menjalankan LLM secara lokal di infrastruktur pengguna [22]. *Platform* mengkuantisasi model secara otomatis agar sesuai sumber daya perangkat keras yang tersedia dan menyediakan API REST yang kompatibel

dengan format OpenAI, sehingga mudah diintegrasikan ke aplikasi Python.

Alasan memilih Ollama bukan sekadar teknis. Kode sumber yang dianalisis dalam penelitian ini bersifat rahasia milik institusi. Mengirimkannya ke API *cloud* bukan pilihan yang bisa diterima. Dengan Ollama, semua inferensi terjadi di mesin lokal tanpa data keluar dari lingkungan organisasi [22], sejalan dengan prinsip ISO/IEC 27001:2022 [12] dan merupakan persyaratan eksplisit dari kedua institusi yang menjadi studi kasus penelitian ini.

BAB III

METODOLOGI PENELITIAN

Bab ini menjelaskan langkah-langkah yang diambil dalam penelitian ini untuk mencapai tujuan yang ditetapkan dalam Subbab 1.3. Cakupannya meliputi alat dan bahan yang digunakan, metode penelitian, arsitektur sistem, alur data *pipeline*, desain *prompt* LLM, pemetaan kontrol ISO/IEC 27001:2022, rancangan eksperimen komparasi model, dan metode evaluasi.

3.1 Alat dan Bahan Penelitian

3.1.1 Perangkat Keras

Seluruh proses pengembangan dan pengujian dalam penelitian ini dilakukan pada satu unit laptop HP Victus 16-r0xxx dengan spesifikasi sebagai berikut:

- Sistem operasi: Windows 11 Home Single Language 64-bit (Build 26200)
- Prosesor: Intel Core i7-13700HX, 24 inti logis, kecepatan dasar 2,1 GHz
- Memori: 16.384 MB RAM
- GPU diskrit: NVIDIA GeForce RTX 4060 dengan VRAM 8 GB
- GPU terintegrasi: Intel UHD Graphics
- Penyimpanan: NVMe SSD 1 TB

Alasan penggunaan spesifikasi ini bukan sekadar ketersediaan. Model LLM yang dijalankan secara lokal melalui Ollama membutuhkan VRAM yang cukup agar inferensi bisa berjalan sepenuhnya di GPU. RTX 4060 dengan VRAM 8 GB memungkinkan kelima model yang dievaluasi (masing-masing 6–8 miliar parameter) dijalankan pada kuantisasi Q4_0 tanpa harus memindahkan sebagian perhitungan ke RAM atau CPU. Spesifikasi ini juga merepresentasikan perangkat keras kelas menengah yang realistis dimiliki oleh institusi atau pengembang individual, sehingga hasil pengujian tidak hanya berlaku di lingkungan riset yang ideal.

3.1.2 Perangkat Lunak

- **Python 3.x:** Bahasa pemrograman utama untuk membangun *pipeline* yang menghubungkan semua modul. Dipilih karena kemudahannya memanggil perintah eksternal via *subprocess* dan ketersediaan pustaka `requests` untuk berkomunikasi dengan API Ollama.
- **Rector [7]:** Alat transformasi kode berbasis AST untuk PHP, dikonfigurasi secara

dinamis sesuai versi target yang dipilih.

- **Semgrep** [9]: Alat analisis statis untuk pemindaian kerentanan. Dijalankan dengan `ruleset p/php` dan `p/owasp-top-ten` [5].
- **PHPStan** [10]: Alat analisis statis untuk validasi tipe dan kompatibilitas kode PHP, dijalankan pada level 5.
- **Ollama** [22]: *Platform* untuk menjalankan LLM secara lokal, diakses via API REST di `localhost:11434`. Ollama dipilih dibandingkan alternatif lain seperti LM Studio dan llama.cpp karena tiga alasan. Pertama, Ollama menyediakan API REST yang kompatibel dengan format OpenAI sehingga mudah diintegrasikan ke skrip Python tanpa pustaka tambahan. Kedua, manajemen model dilakukan melalui satu perintah `ollama pull` tanpa konfigurasi manual, sehingga eksperimen dapat direproduksi dengan mudah. Ketiga, Ollama mendukung kuantisasi Q4_0 secara otomatis yang memungkinkan kelima model berjalan dalam VRAM 8 GB tanpa pengaturan tambahan. LM Studio menyediakan antarmuka grafis yang tidak diperlukan untuk otomatisasi berbasis skrip, sementara llama.cpp memerlukan kompilasi manual dan tidak memiliki API REST bawaan. Secara teknis, Ollama memuat bobot model dalam format GGUF ke dalam VRAM GPU menggunakan *backend* llama.cpp. Proses inferensi berjalan sepenuhnya di GPU: setiap token keluaran dihasilkan melalui operasi perkalian matriks pada bobot model yang sudah dikuantisasi tanpa melibatkan server eksternal. Pada RTX 4060 dengan VRAM 8 GB, kelima model yang diuji menghasilkan rata-rata 20–40 token per detik pada kuantisasi Q4_0. Ollama menyimpan model yang sudah dimuat di VRAM selama sesi aktif sehingga latensi *cold start* (waktu muat model pertama kali ke VRAM) hanya terjadi sekali per sesi eksperimen
- **PHP 8.5.1**: Interpreter PHP untuk validasi sintaks menggunakan `php -l`.
- **Composer**: Manajer dependensi PHP untuk instalasi Rector dan PHPStan.
- **Visual Studio Code** dengan ekstensi Claude Code sebagai IDE utama selama pengembangan.

3.1.3 Model LLM yang Dievaluasi

Lima model LLM *open source* dievaluasi, semuanya dijalankan secara lokal melalui Ollama tanpa *fine-tuning*. Pemilihan model didasarkan pada dua kriteria: model harus dapat dijalankan di GPU dengan VRAM 8 GB pada kuantisasi Q4_0 [22], dan kemampuan pemahaman kodenya harus terdokumentasi dalam literatur ilmiah [17] [18] [19] [20] [21].

- DeepSeek Coder 6.7b [17]: Model spesialis kode, dilatih pada 2 triliun token dengan

87% kode sumber.

- Qwen2.5-Coder 7b [18]: Model kode dari Alibaba, dilatih pada 5,5 triliun token kode.
- CodeLlama 7b [19]: Model berbasis Llama 2 dari Meta AI untuk tugas pemrograman.
- Mistral 7b [20]: Model tujuan umum dari Mistral AI dengan arsitektur *grouped-query attention*.
- Llama 3.1 8b [21]: Model dari seri Llama 3 yang dilatih pada lebih dari 15 triliun token multibahasa.

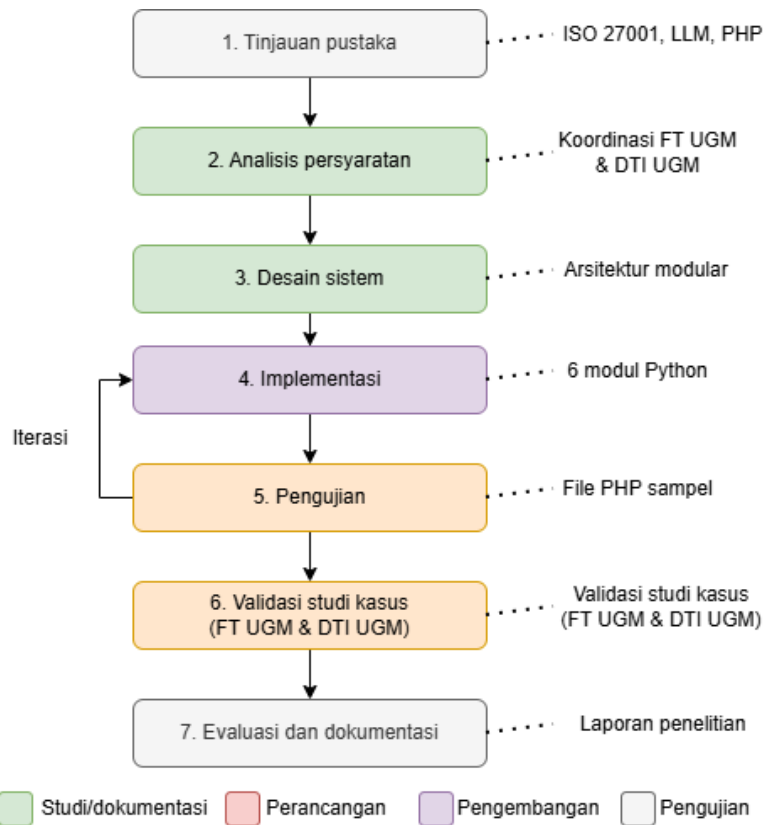
3.1.4 Bahan Penelitian

- **Dokumen ISO/IEC 27001:2022** [12]: Acuan utama untuk pemetaan kontrol keamanan.
- **Kode sumber PHP dari FT UGM**: Kumpulan *file* PHP dari aplikasi *Dashboard* TCK berbasis CodeIgniter 3.x dengan PHP 7.4.9. Bersifat rahasia dan hanya digunakan untuk analisis statis tanpa akses ke basis data produksi.
- **Kode sumber PHP dari DTI UGM**: Kumpulan *file* PHP dari aplikasi *Computer-Based Test* (CBT) berbasis CodeIgniter 3.x dengan persyaratan PHP ≥ 7.0 . Bersifat rahasia dengan perlakuan privasi data yang sama dengan dataset FT UGM.
- **Dokumentasi resmi PHP** [3]: Referensi untuk memverifikasi kebenaran konversi Rector.
- **Registri aturan Semgrep** [9]: Aturan publik `p/php` dan `p/owasp-top-ten` untuk pemindaian keamanan.

3.2 Metode Penelitian

Penelitian ini menggunakan metodologi Penelitian dan Pengembangan (*Research and Development*, R&D) dengan model prototipe. Pilihan ini didasarkan pada fakta bahwa penelitian menghasilkan produk perangkat lunak nyata, bukan hanya analisis teoritis. Model prototipe dipilih karena dalam praktiknya banyak keputusan desain baru bisa dibuat setelah melihat perilaku nyata sistem, bukan dari perencanaan di atas kertas saja. Ini terutama berlaku untuk format *prompt* AI dan cara menangani kode yang tidak bisa diotomatisasi sepenuhnya.

Gambar 3.1 menunjukkan alur penelitian yang terdiri dari tujuh tahapan.



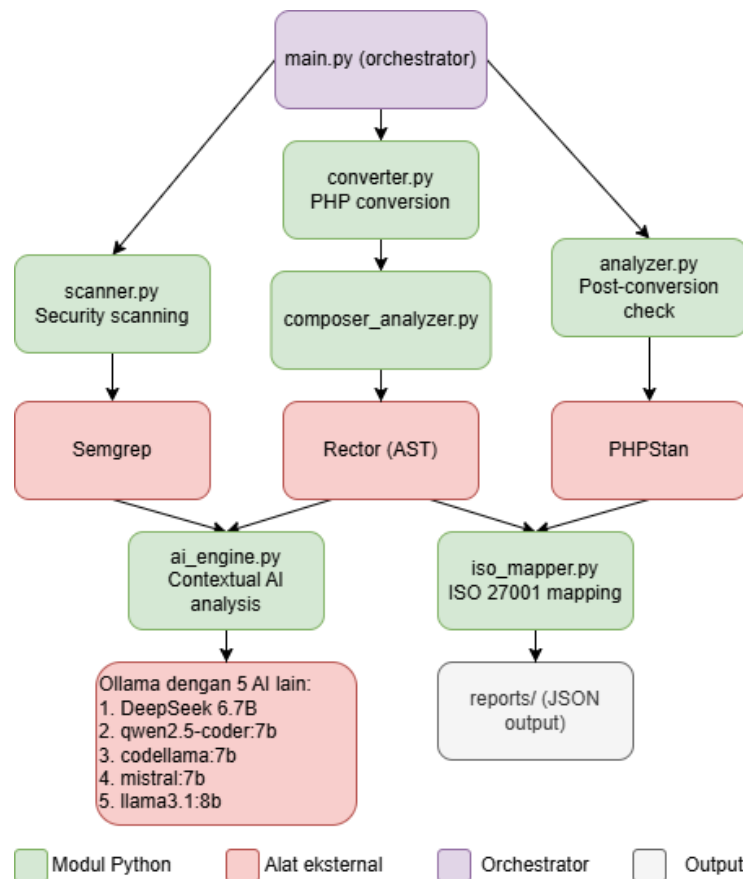
Gambar 3.1. Alur penelitian dengan model prototipe

1. **Tinjauan pustaka:** Mempelajari permasalahan migrasi PHP lama, persyaratan ISO/IEC 27001:2022 [12], kemampuan model LLM *open source*, dan alat analisis statis yang relevan. Koordinasi awal dengan FT UGM dan DTI UGM dilakukan pada tahap ini.
2. **Analisis persyaratan:** Mendefinisikan persyaratan fungsional dan non-fungsional berdasarkan hasil tinjauan pustaka dan koordinasi dengan FT UGM dan DTI UGM, termasuk menentukan kontrol ISO yang dicakup dan persyaratan privasi data.
3. **Desain sistem:** Merancang arsitektur *pipeline* secara modular dan menentukan tanggung jawab setiap modul.
4. **Implementasi:** Membangun keenam modul Python satu per satu. Setiap modul diuji secara terpisah sebelum diintegrasikan.
5. **Pengujian:** Menguji prototipe dengan *file* PHP sampel. Jika ditemukan masalah, proses kembali ke tahap implementasi (iterasi).
6. **Validasi studi kasus:** Pengujian akhir menggunakan kode sumber asli FT UGM dan DTI UGM setelah prototipe stabil.
7. **Evaluasi dan dokumentasi:** Menganalisis hasil dan menyusun laporan penelitian.

3.3 Arsitektur Sistem

Sistem terdiri dari enam modul Python yang masing-masing menangani satu langkah dalam proses konversi dan analisis keamanan. `main.py` bertindak sebagai orkestrator yang mengontrol urutan eksekusi dan mengalirkan data antar modul. Arsitektur modular memungkinkan setiap bagian diuji, diperbaiki, atau diganti secara independen.

Gambar 3.2 menunjukkan hubungan antar modul.



Gambar 3.2. Arsitektur sistem *pipeline* dengan enam modul Python

3.3.1 Scanner (`scanner.py`)

Modul pertama yang berjalan. Scanner menjalankan Semgrep [9] pada direktori `input/` sebelum konversi dimulai untuk menetapkan *baseline* keamanan kode asli. Hasil pemindaian dikelompokkan berdasarkan jenis kerentanan (injeksi SQL, XSS, *path traversal*, injeksi perintah, fungsi usang, kriptografi lemah) dan diberi tingkat prioritas. Data ini menjadi pembanding untuk mengukur perubahan keamanan setelah konversi.

3.3.2 Converter (converter.py)

Modul ini menggunakan Rector [7] untuk transformasi kode PHP berbasis AST. Argumen CLI `-target-php` menentukan versi target; nilai yang diterima adalah "7.4", "8.0", "8.1", "8.2", atau "8.3". Sebelum Rector dijalankan, seluruh *file* PHP dari `input/` disalin dahulu ke `output/`. Rector kemudian memodifikasi *file* di `output/` di tempat. *File* yang gagal dikonversi tetap ada di `output/` sebagai salinan kode asli dan statusnya dicatat `FAILED` di laporan; direktori `input/` tidak pernah disentuh. Konfigurasi Rector dibuat sebagai *tempfile* saat *runtime* dan dihapus otomatis setelahnya.

Tabel 3.1. Pemetaan argumen `-target-php` ke konfigurasi Rector

Nilai <code>-target-php</code>	LevelSetList yang diterapkan	Cakupan rule kumulatif
7.4	UP_TO_PHP_74	PHP 5.2, 5.3, 5.4, 5.5, 5.6, 7.0, 7.1, 7.2, 7.3, 7.4
8.0	UP_TO_PHP_80	Semua rule 7.4 + PHP 8.0
8.1	UP_TO_PHP_81	Semua rule 8.0 + PHP 8.1
8.2	UP_TO_PHP_82	Semua rule 8.1 + PHP 8.2
8.3	UP_TO_PHP_83	Semua rule 8.2 + PHP 8.3

Karena *ruleset*-nya kumulatif, kode PHP 5.x pun bisa diproses langsung menuju PHP 8.3 dalam satu langkah. Transformasi seperti `var` menjadi `public`, `array()` menjadi `[]`, dan penghapusan modifier `/e` pada `preg_replace()` ditangani sepenuhnya. Yang tidak bisa ditangani adalah `mysql_*` karena butuh penulisan ulang semantik, bukan sekadar penggantian sintaks.

3.3.3 Analyzer (analyzer.py)

Setelah konversi, Analyzer menjalankan PHPStan [10] pada direktori `output/` untuk memverifikasi kompatibilitas kode dengan versi PHP target. Level 5 dipilih sebagai keseimbangan antara keketatan dan jumlah *false positive* yang wajar untuk kode warisan. Setiap temuan PHPStan dikategorikan sebagai `ERROR` atau `WARNING`, lalu dikaitkan dengan kontrol ISO/IEC 27001:2022 yang relevan.

3.3.4 AI Engine (ai_engine.py)

Modul ini menerima temuan dari Semgrep *pre-scan* sebagai masukan, bukan dari PHPStan dan bukan dari *post-scan*. Hanya temuan dengan prioritas 1–3 yang dikirim ke AI Engine: SQL Injection (prioritas 1), XSS (prioritas 2), dan fungsi usang (prioritas 3). Temuan prioritas 4 ke atas seperti *hardcoded credentials* dan *path traversal* tidak diproses.

Untuk setiap temuan yang lolos filter, modul mengirimkan konteks dan potongan kode ke model LLM melalui endpoint `/api/chat` Ollama [22] di `localhost:11434`. Kalau model tidak menghasilkan blok kode PHP sama sekali, respons tetap dicatat dengan *placeholder* `// Manual review required`, bukan di-skip. Jika Ollama tidak tersedia, *pipeline* tetap berjalan tanpa komponen AI. Desain *prompt* yang digunakan dibahas dalam Subbab 3.5.

3.3.5 ISO Mapper (`iso_mapper.py`)

Modul ini menerima semua hasil dari Scanner, Analyzer, dan AI Engine sebagai objek Python *in-memory* langsung dari `main.py`, lalu memetakannya ke kontrol ISO/IEC 27001:2022 [12] secara berbasis aturan. Tidak ada pembacaan *file* JSON. Pemetaan berbasis aturan dipilih agar hasilnya deterministik dan dapat diaudit. Detail pemetaan dibahas dalam Subbab 3.6.

3.3.6 Main (`main.py`)

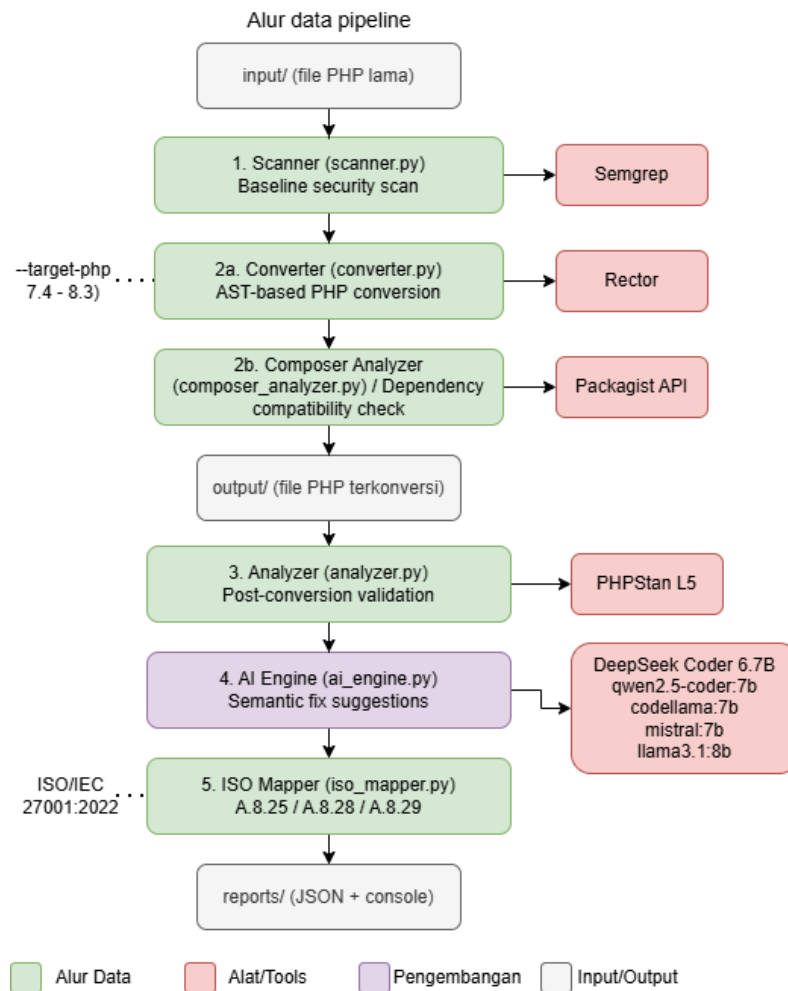
File `main` adalah orkestrator yang mengontrol seluruh *pipeline*. Menerima direktori *input* dan versi PHP target sebagai argumen CLI, menjalankan keenam modul secara berurutan, dan menghasilkan laporan JSON di direktori `reports/`. Ringkasan hasil ditampilkan di konsol menggunakan pustaka Rich.

3.3.7 Composer Analyzer (`composer_analyzer.py`)

Modul ini membaca berkas `composer.json` pada direktori *root* repositori dan memeriksa kompatibilitas setiap dependensi terhadap PHP 8.x menggunakan data dari Packagist. Jika berkas `composer.json` tidak ditemukan, modul dilewati secara otomatis tanpa menghentikan *pipeline*. Setiap dependensi diklasifikasikan sebagai `SAFE` (versi yang dikonfigurasi mendukung PHP 8.x secara deklaratif) atau `MANUAL REVIEW` (kompatibilitas tidak dapat diverifikasi secara otomatis).

3.4 Alur Data Pipeline

Gambar 3.3 menunjukkan alur data lengkap dari masukan hingga laporan akhir.



Gambar 3.3. Alur data *pipeline* dari *input* hingga laporan akhir

Alurnya cukup linear. *File* PHP ditempatkan di `input/`. Scanner memindai dengan Semgrep untuk menetapkan *baseline*. Converter menyalin semua *file* ke `output/` lalu menjalankan Rector dengan versi target yang ditentukan; `input/` tidak tersentuh. Composer Analyzer memeriksa kompatibilitas dependensi `composer.json` terhadap PHP 8.x secara paralel, atau dilewati otomatis jika berkas tidak ditemukan. Analyzer menjalankan PHPStan level 5 pada `output/`. AI Engine mengambil temuan Semgrep prioritas 1–3 dan mengirimkannya ke model LLM. ISO Mapper mengumpulkan semua temuan dan memetakannya ke kontrol ISO. `main.py` menggabungkan semuanya menjadi laporan JSON dan menampilkan ringkasan di konsol.

3.5 Desain Prompt LLM

Desain *prompt* adalah salah satu keputusan metodologis yang paling berpengaruh terhadap kualitas respons model. Tiga tujuan harus dipenuhi sekaligus: menghasilkan kode perbaikan yang valid secara sintaksis, menghasilkan skor kepercayaan yang dapat dibaca otomatis oleh *pipeline*, dan dapat dijalankan secara *zero-shot* tanpa contoh mengingat tidak tersedianya *dataset* berlabel.

3.5.1 Struktur Prompt

Prompt terdiri dari dua bagian yang dikirim melalui endpoint `/api/chat`: *system message* dan *user message*. *System message*:

```
You are a PHP security expert and migration specialist.
You analyze PHP code for security vulnerabilities and provide
secure PHP 8.x replacements.
Always follow output format instructions exactly as given.
```

User message untuk setiap temuan:

```
TASK: Analyze the following PHP code for a [vulnerability_type]
vulnerability and provide a secure PHP 8.x replacement.
ISO/IEC 27001:2022 Controls: [relevant_controls]
Context: File: [filename], line [line_number],
        Semgrep rule: [rule_id], Finding: [description]
VULNERABLE CODE:
```php
[code_snippet]
```

Respond in English:
1. Explain the vulnerability and its security risk.
2. Provide the corrected PHP 8.x code in a ```php code block.
After your explanation and code fix, write exactly this on the
last line:
CONFIDENCE: [number between 0 and 100]
Example last line: CONFIDENCE: 85
```

3.5.2 Keputusan Desain Prompt

Penyertaan kontrol ISO/IEC 27001:2022. Kontrol ISO yang relevan disertakan di setiap *prompt*. Ini bukan konteks dekoratif; tujuannya mendorong model

menghasilkan saran perbaikan yang mempertimbangkan kepatuhan standar, bukan sekadar memperbaiki sintaks.

Instruksi format di akhir pesan. Format `CONFIDENCE: [0-100]` diletakkan di bagian akhir dengan instruksi eksplisit termasuk contoh konkret. Model kecil cenderung lebih patuh pada instruksi yang dekat dengan akhir pesan.

Parsing skor berlapis. Karena tidak semua model mengikuti format yang diminta, `ai_engine.py` menggunakan enam pola *regex* berurutan untuk mengekstraksi skor: (0) tag XML `<confidence>`, (1) bilangan bulat dekat kata “confidence”, (2) desimal `0.x` dekat “confidence”, (3) persentase dekat “confidence”, (4) desimal *bare* di mana saja, dan (5) frasa bahasa alami seperti “not sure” yang dipetakan ke nilai tetap. Jika semua gagal, dikembalikan nilai *default* 0,0.

Temperature 0,1. Dipilih untuk meminimalkan variasi acak antar percobaan. Setiap model dijalankan tiga kali per temuan dengan *prompt* identik, sehingga perbedaan hasil antar *run* mencerminkan variabilitas model yang sebenarnya.

Zero-shot tanpa contoh. Tidak ada contoh perbaikan kode disertakan karena penelitian ingin mengevaluasi kemampuan bawaan model dalam kondisi yang merepresentasikan penggunaan nyata. Ini juga memastikan semua model dibandingkan pada kondisi yang setara.

Skor kepercayaan sebagai metrik sekunder. Skor ini adalah penilaian mandiri dari model dan tidak bisa diverifikasi secara independen. Dalam analisis Bab IV, skor kepercayaan adalah metrik sekunder; metrik primer adalah validitas sintaks kode yang dihasilkan, diukur dengan `php -l`.

3.6 Desain Pemetaan ISO/IEC 27001:2022

Pemetaan temuan ke kontrol ISO/IEC 27001:2022 dilakukan secara berbasis aturan (*rule-based*) di dalam `iso_mapper.py`. Data diterima langsung sebagai objek Python *in-memory* dari `main.py`, tidak dibaca dari *file* JSON. Pilihan desain berbasis aturan ini disengaja agar hasilnya deterministik dan bisa diaudit, berbeda dari pendekatan berbasis AI yang hasilnya bisa berubah antar inferensi. Ada enam kontrol yang dipetakan, sebagaimana ditunjukkan pada Tabel 3.2.

Status akhir setiap kontrol ditentukan berdasarkan ada tidaknya temuan terbuka setelah *pipeline* selesai. `COMPLIANT` berarti tidak ada temuan yang dipetakan ke kontrol tersebut. `NON_COMPLIANT` berarti ada temuan Semgrep aktif. `PARTIAL` berarti hanya ada temuan PHPStan tanpa temuan Semgrep. Perlu dicatat bahwa `COMPLIANT` pada A.5.17 dan A.8.24 bukan berarti kode bebas masalah secara absolut, melainkan *ruleset* yang digunakan tidak menghasilkan temuan yang dipetakan ke kedua kontrol itu pada dataset ini.

Tabel 3.2. Pemetaan sumber temuan ke kontrol ISO/IEC 27001:2022 Annex A

| Kontrol ISO | Judul | Sumber temuan yang dipetakan |
|-------------|--|--|
| A.5.17 | <i>Authentication Information</i> | Semgrep: <i>hardcoded credentials</i> |
| A.8.24 | <i>Use of Cryptography</i> | Semgrep: <code>md5()</code> / <code>sha1()</code> untuk kata sandi |
| A.8.25 | <i>Secure Development Lifecycle</i> | Semgrep (semua) + PHPStan ERROR/WARNING |
| A.8.26 | <i>Application Security Requirements</i> | Semgrep: SQL Injection, XSS |
| A.8.28 | <i>Secure Coding</i> | Semgrep (semua) + PHPStan ERROR |
| A.8.29 | <i>Security Testing in Development</i> | PHPStan ERROR/WARNING |

3.6.1 Mekanisme Klasifikasi Temuan

Pemetaan dari hasil temuan Semgrep ke kontrol ISO dilakukan dengan fungsi `_classify()` di `scanner.py`. Fungsi ini memeriksa `rule_id` dan `message` dari setiap temuan Semgrep menggunakan pencocokan *substring* (bukan ekspresi reguler) terhadap daftar kata kunci yang telah ditetapkan. Hasilnya adalah penugasan `vuln_type`, daftar `iso_controls`, dan nilai prioritas. Tabel 3.3 menunjukkan pemetaan kata kunci ke jenis kerentanan dan kontrol ISO yang ditugaskan.

Tabel 3.3. Pemetaan kata kunci `rule_id` Semgrep ke jenis kerentanan dan kontrol ISO/IEC 27001:2022

| Kata Kunci (<code>rule_id/message</code>) | Jenis Kerentanan | Kontrol ISO | Prioritas |
|---|----------------------|----------------|-----------|
| <code>sql, sqli, injection</code> | SQL Injection | A.8.26, A.8.28 | 1 |
| <code>xss, cross-site</code> | XSS | A.8.26, A.8.28 | 2 |
| <code>deprecated, mysql_</code> | Fungsi Usang | A.8.25, A.8.28 | 3 |
| <code>hardcoded, secret, password</code> | Kredensial Hardcoded | A.5.17, A.8.28 | 4 |
| <code>path, traversal, ssrf</code> | Path Traversal/SSRF | A.8.28, A.8.29 | 4 |
| <code>md5, sha1, weak.*crypt</code> | Kriptografi Lemah | A.8.24, A.8.28 | 5 |

Pendekatan *keyword matching* ini dipilih secara sengaja dibandingkan pendekatan berbasis AI karena tiga alasan. Pertama, hasilnya deterministik: temuan yang sama selalu menghasilkan pemetaan yang sama tanpa variasi antar inferensi. Kedua, dapat diaudit secara langsung karena seluruh logika pemetaan tersimpan sebagai kode Python yang bisa diperiksa dan diverifikasi. Ketiga, tidak memerlukan koneksi eksternal atau model tambahan, sehingga konsisten dengan persyaratan privasi data yang ditetapkan dalam Subbab 3.9.

3.7 Dataset Studi Kasus

Studi kasus pertama menggunakan 245 *file* PHP dari satu aplikasi web internal berbasis CodeIgniter 3.x yang dikelola oleh FT UGM. Struktur direktorinya mengikuti konvensi standar CodeIgniter 3.x: *application/* untuk kode aplikasi (kontroler, model, *view*, *helper*, konfigurasi, layanan) dan *system/* untuk kode *framework*. Kode diperoleh melalui koordinasi langsung dengan FT UGM dan hanya digunakan untuk analisis statis dalam lingkungan lokal, tanpa akses ke basis data produksi dan tanpa eksekusi kode.

Hanya *file* .php yang diproses; *file* lain seperti .html, .js, dan .css tidak disalin ke *output/* dan tidak dianalisis. Dataset mencerminkan karakteristik kode warisan yang umum: pencampuran logika bisnis, akses basis data, dan presentasi dalam *file* yang sama, serta beberapa pola kode PHP 5.x yang masih ada meski aplikasi secara keseluruhan berjalan di atas PHP 7.x.

Untuk eksperimen komparasi LLM, digunakan 11 temuan dari Semgrep *pre-scan*: 1 SQL Injection dan 10 SSRF (*Server-Side Request Forgery*). Ini adalah seluruh populasi temuan *pre-scan*, bukan subset yang dipilih secara subjektif. Studi kasus kedua menggunakan 183 *file* PHP kustom dari aplikasi CBT milik DTI UGM, dengan mengeksklusi direktori *libraries/*, *third_party/*, dan *tcpdf/*. Target konversi adalah PHP 8.3. Karena tidak ada berkas *composer.json* pada tingkat aplikasi, langkah analisis dependensi dilewati secara otomatis untuk studi kasus ini.

3.8 Rancangan Eksperimen Komparasi Model LLM

Eksperimen komparasi dirancang dengan kondisi yang sepenuhnya terkontrol dan dilakukan terpisah dari pengujian *pipeline* utama menggunakan skrip evaluasi khusus.

3.8.1 Kondisi Eksperimen

- **Input:** 11 temuan Semgrep dari *pre-scan* dataset FT UGM
- **Jumlah percobaan:** 3 *run* per temuan per model (33 percobaan per model, 165 total)
- **Temperature:** 0,1 untuk semua model
- **Prompt:** identik untuk semua model dan semua percobaan (*zero-shot*)
- **Model:** dijalankan lokal melalui Ollama tanpa modifikasi

3.8.2 Metrik Evaluasi

Berikut definisi formal metrik yang digunakan dalam penelitian ini.

Tingkat Konversi Rector (*Conversion Rate*, CR) mengukur persentase *file* PHP yang berhasil dikonversi:

$$CR = \frac{f_{success}}{f_{total}} \times 100\% \quad (3-1)$$

di mana $f_{success}$ adalah jumlah *file* yang berhasil dikonversi dan f_{total} adalah total *file* PHP dalam direktori `input/`.

Format Compliance Rate (FCR) mengukur persentase respons model yang memenuhi ketiga elemen format *prompt* sekaligus:

$$FCR = \frac{r_{compliant}}{r_{total}} \times 100\% \quad (3-2)$$

di mana $r_{compliant}$ adalah jumlah respons yang patuh format dan r_{total} adalah total percobaan per model (33 per model). Metrik ini mencerminkan kemampuan model mengikuti instruksi, bukan kualitas kodenya.

Syntax Validity Rate (SVR) mengukur persentase blok kode PHP yang diekstraksi dan lolos `php -l`:

$$SVR = \frac{c_{valid}}{c_{extracted}} \times 100\% \quad (3-3)$$

di mana c_{valid} adalah jumlah blok kode yang lolos pemeriksaan sintaks dan $c_{extracted}$ adalah total blok kode yang berhasil diekstraksi dari respons model. Ini adalah metrik primer karena hasilnya konkret dan dapat diverifikasi secara independen.

Security Finding Delta (ΔF) mengukur perubahan jumlah temuan keamanan antara *pre-scan* dan *post-scan*:

$$\Delta F = F_{post} - F_{pre} \quad (3-4)$$

Nilai $\Delta F < 0$ menunjukkan pengurangan temuan, $\Delta F = 0$ berarti tidak ada perubahan, dan $\Delta F > 0$ tidak otomatis diinterpretasikan sebagai regresi karena bisa mencerminkan kerentanan laten yang baru terlihat setelah transformasi kode.

Mean Confidence Score ($\bar{C}_m$) mengukur rata-rata skor kepercayaan yang dilaporkan sendiri oleh model m :

$$\bar{C}_m = \frac{1}{n} \sum_{i=1}^n c_i \quad (3-5)$$

di mana n adalah jumlah *run* dan c_i adalah skor kepercayaan pada *run* ke- i . Untuk mengukur konsistensi antar *run*, digunakan variansi:

$$\sigma_m^2 = \frac{1}{n} \sum_{i=1}^n (c_i - \bar{C}_m)^2 \quad (3-6)$$

Model dengan σ_m^2 rendah menunjukkan perilaku yang lebih stabil dan dapat diandalkan. Perlu dicatat bahwa $\bar{C}_m$ adalah metrik sekunder; implikasinya terhadap kualitas kode dibahas di Bab IV.

3.8.3 Interpretasi Trade-off Antar Metrik

Kedua metrik utama FCR dan SVR bisa memberikan gambaran yang berbeda dan perlu diinterpretasikan bersama. Model dengan FCR tinggi tapi SVR rendah menunjukkan kemampuan mengikuti instruksi format namun kualitas kode yang buruk. Sebaliknya, model dengan FCR rendah tapi SVR tinggi mengabaikan instruksi format namun menghasilkan kode yang berkualitas baik. Kedua kondisi ini punya implikasi praktis yang berbeda dan akan dibahas di Bab IV.

3.9 Keamanan dan Privasi Data

Kode PHP dari FT UGM dan DTI UGM adalah aset institusi yang tidak boleh dikirimkan ke layanan eksternal. Kendala inilah yang menentukan arsitektur AI dalam penelitian ini. Daripada menggunakan API LLM berbasis *cloud*, penelitian ini menggunakan Ollama [22] sebagai *runtime* lokal sehingga seluruh inferensi terjadi di mesin pengembangan.

Tidak ada data yang dikirimkan ke *server* eksternal. Seluruh proses analisis berlangsung secara statis: tidak ada koneksi ke basis data produksi FT UGM, tidak ada eksekusi kode PHP. Keputusan ini sejalan dengan ISO/IEC 27001:2022 [12] yang mensyaratkan pemisahan lingkungan pengembangan dan pengendalian akses terhadap aset informasi sensitif.

3.10 Metode Pengujian dan Evaluasi

Pengujian dilakukan dengan pendekatan *black-box*, fokus pada *output* setiap modul bukan detail implementasi internal. Ada lima metrik utama:

1. **Tingkat konversi Rector:** Persentase *file* PHP yang berhasil dikonversi tanpa kesalahan fatal Rector, dihitung per *file* dan secara keseluruhan.
2. **Validitas sintaks pasca-konversi:** Persentase *file* hasil konversi yang lolos `php -l`. Diukur secara objektif.
3. **Perbedaan temuan keamanan:** Selisih jumlah temuan Semgrep antara *pre-scan* dan *post-scan*. Temuan baru setelah konversi tidak otomatis diinterpretasikan

sebagai regresi keamanan; bisa jadi merupakan kerentanan laten yang sebelumnya tersembunyi dan baru terlihat setelah transformasi kode.

4. **Performa model LLM:** Diukur menggunakan *format compliance rate* (sekunder) dan *syntax validity rate* (primer) sebagaimana dijelaskan dalam Subbab 3.8.2.
5. **Status kepatuhan ISO/IEC 27001:2022:** Status per kontrol (COMPLIANT, PARTIAL, atau NON_COMPLIANT) berdasarkan logika pemetaan dalam Subbab 3.6.

Untuk hasil PHPStan, jumlah kesalahan yang tinggi tidak secara langsung diinterpretasikan sebagai kegagalan *pipeline*. Kesalahan PHPStan pada kode warisan yang baru dimigrasi sebagian besar mencerminkan *technical debt* yang sudah ada sebelum migrasi, bukan akibat dari proses konversi. Perbedaan ini dibahas lebih rinci di Bab IV.

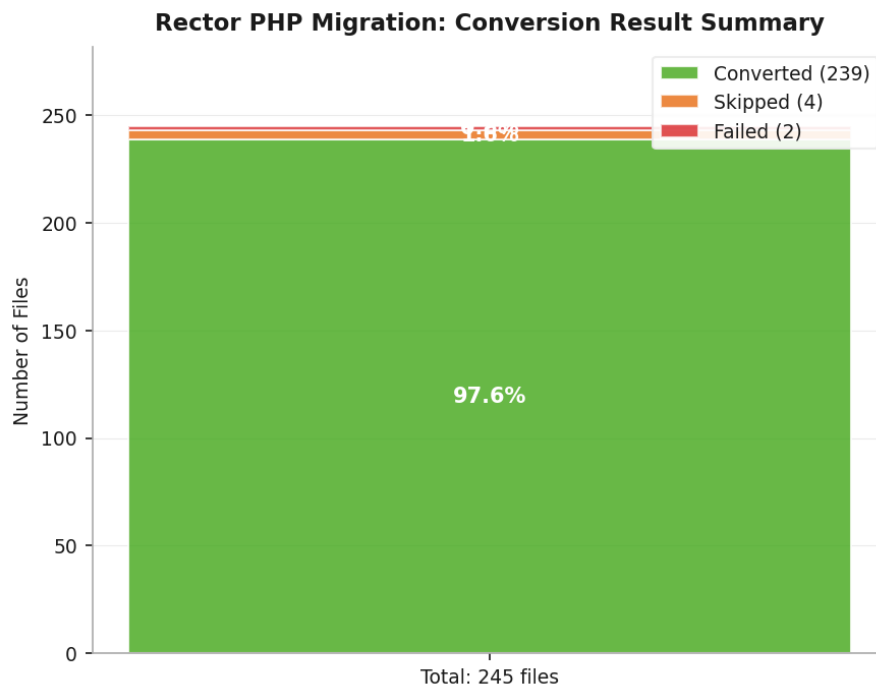
BAB IV

HASIL DAN DISKUSI

Bab ini menyajikan hasil implementasi *pipeline* pada dua studi kasus: kode PHP milik FT UGM (Dashboard TCK) dan kode PHP milik DTI UGM (Sistem CBT), beserta analisis perbandingan keduanya. Pembahasan studi kasus pertama mencakup: hasil konversi Rector, hasil pemindaian keamanan Semgrep sebelum dan sesudah konversi, hasil validasi PHPStan, status kepatuhan ISO/IEC 27001:2022, analisis dependensi *composer*, hasil komparasi lima model LLM dengan *prompt* identik (Kondisi A), hasil komparasi dengan *prompt* yang dioptimasi per model (Kondisi B), optimasi parameter Qwen2.5-Coder (Kondisi C), serta diskusi dan keterbatasan penelitian. Bagian penutup menyajikan perbandingan hasil kedua studi kasus secara menyeluruh.

4.1 Hasil Konversi Rector

Rector dijalankan terhadap 245 *file* PHP dari dataset FT UGM dengan target versi PHP 8.3. Dari 245 *file* tersebut, 239 *file* berhasil dikonversi, 4 *file* dilewati (*skipped*), dan 2 *file* gagal diproses. Tingkat konversi keseluruhan ($CR = 97,6\%$), sebagaimana ditunjukkan pada Gambar 4.4.



Source: pipeline_result_20260425_120423.json | Skripsi Muhammad Farrel Akbar

Gambar 4.4. Ringkasan hasil konversi Rector terhadap 245 *file* PHP dataset FT UGM

Karena dataset mengandung campuran kode PHP 5.x dan PHP 7.x, Rector menerapkan rantai *ruleset* kumulatif dari UP_TO_PHP_53 hingga UP_TO_PHP_83 secara otomatis dalam satu proses. Ini membuktikan bahwa *pipeline* dapat menangani kode dari berbagai era PHP sekaligus tanpa konfigurasi tambahan.

4.1.1 Contoh Transformasi Rector

Untuk memperlihatkan jenis-jenis perubahan yang dilakukan Rector secara konkret, berikut tiga contoh transformasi yang ditemukan pada dataset FT UGM.

Transformasi 1: Array sintaks pendek (**ArrayShortSyntaxRector**)

Pada `system/core/Security.php`, deklarasi *array* menggunakan sintaks lama diubah ke sintaks pendek PHP 5.4:

```
// Sebelum (PHP 7.x)
public $filename_bad_chars = array(
    '../', '<!--', '-->', '<', '>',
    '"', "'", '&', '$', '#',
);
protected $_never_allowed_str = array(
    'document.cookie' => '',
    'document.write'  => '',
);

// Sesudah (PHP 8.3)
public $filename_bad_chars = [
    '../', '<!--', '-->', '<', '>',
    '"', "'", '&', '$', '#',
];
protected $_never_allowed_str = [
    'document.cookie' => '',
    'document.write'  => '',
];
```

Transformasi 2: Operator *null coalescing* (**NullCoalescingRector**)

Pada `system/helpers/url_helper.php`, pola pengecekan tiga bagian `isset()` diubah ke operator `??` yang lebih ringkas:

```
// Sebelum (PHP 7.x)
$params[$key] = isset($attributes[$key])
    ? $attributes[$key] : $val;
```

```
// Sesudah (PHP 8.3)
$atts[$key] = $attributes[$key] ?? $val;
```

Transformasi 3: Fungsi `str_contains()` (`StrContainsRector`)

Pada `system/helpers/url_helper.php`, pola pencarian substring dengan `strpos()` diubah ke fungsi `str_contains()` yang diperkenalkan di PHP 8.0. Rector juga menambahkan *cast* (`string`) karena hasil dari `$_SERVER['SERVER_SOFTWARE']` bertipe `mixed` di PHP 8.x:

```
// Sebelum (PHP 7.x)
if ($method === 'auto'
    && isset($_SERVER['SERVER_SOFTWARE'])
    && strpos($_SERVER['SERVER_SOFTWARE'],
        'Microsoft-IIS') !== FALSE)

// Sesudah (PHP 8.3)
if ($method === 'auto'
    && isset($_SERVER['SERVER_SOFTWARE'])
    && str_contains(
        (string) $_SERVER['SERVER_SOFTWARE'],
        'Microsoft-IIS'))
```

Ketiga transformasi ini mencerminkan kategori perubahan yang paling umum dilakukan Rector: modernisasi sintaks, penggunaan API PHP yang lebih baru, dan penambahan informasi tipe yang dibutuhkan PHP 8.x.

4.1.2 File yang Gagal Dikonversi

Kedua *file* yang gagal dikonversi mengalami *internal error* yang sama, yaitu pada fungsi `FiberNodeScopeResolver::callNodeCallback()` yang menerima argumen bertipe `null`, padahal seharusnya bertipe `PhpParser\Node`. Permasalahan ini merupakan *bug* pada versi Rector yang digunakan, bukan berasal dari kode sumber FT UGM.

File pertama, yaitu `captcha_helper.php`, memiliki pola percabangan yang mengembalikan tipe data berbeda, yaitu `GdImage` dan `resource`, bergantung pada hasil evaluasi `function_exists()`. Rector menggunakan *Fiber* mekanisme konkurensi ringan yang diperkenalkan di PHP 8.1 sebagai unit eksekusi untuk menganalisis setiap *node* AST secara terpisah selama proses transformasi [3]. Dalam kondisi ini, PHPStan *Fiber* yang digunakan oleh Rector kehilangan referensi terhadap *node* AST ketika menganalisis percabangan dengan perbedaan tipe tersebut.

File kedua, yaitu `form_helper.php`, mengandung lebih dari 15 definisi fungsi kondisional yang dibungkus dalam konstruksi `if (!function_exists(...))` pada ruang lingkup *file* yang sama. Banyaknya *fiber* analisis yang berjalan secara paralel untuk setiap fungsi menyebabkan potensi kondisi *race*, di mana salah satu *callback* menerima *node* yang telah selesai diproses oleh *fiber* lain.

Kedua *file* tersebut tetap disertakan dalam direktori `output/` sebagai salinan kode asli, sehingga proses konversi tetap berjalan tanpa kehilangan data.

Setelah proses konversi, seluruh *file* pada direktori `output/` diuji validitas sintaksisnya menggunakan perintah `php -l`. Hasil pengujian menunjukkan tidak adanya *parse error*, sehingga tingkat validitas sintaks pasca-konversi mencapai 100%.

4.2 Hasil Pemindaian Keamanan Semgrep

4.2.1 Pre-Scan: Baseline Keamanan Kode Asli

Pemindaian Semgrep terhadap 241 *file* PHP pada direktori `input/` (dari total 245 *file*, dengan 4 *file* yang dilewati oleh Rector) menghasilkan 11 temuan, yang terdiri atas 1 kerentanan SQL Injection dan 10 kerentanan SSRF (*Server-Side Request Forgery*). Seluruh temuan tersebut terlokalisasi pada satu *file*, yaitu `Output.php`.

Kerentanan SQL Injection teridentifikasi pada baris 768, di mana data dari variabel `$_SERVER['QUERY_STRING']` digunakan secara langsung dalam konstruksi *cache path* tanpa melalui proses sanitasi yang sesuai. Selain itu, terdapat 10 temuan SSRF yang muncul pada beberapa bagian dalam `Output.php`. Temuan ini disebabkan oleh penggunaan nama *file* yang berasal dari *input* pengguna dalam operasi *file system*, sehingga membuka peluang akses terhadap sumber daya yang tidak diinginkan.

Terpusatnya seluruh temuan pada satu *file* mengindikasikan pola yang umum pada kode warisan, di mana modul yang menangani *output* dan *cache* cenderung menjadi titik kerentanan akibat tingginya interaksi dengan data eksternal yang tidak divalidasi secara konsisten.

4.2.2 Post-Scan: Perbandingan Setelah Konversi

Pemindaian Semgrep terhadap direktori `output/` setelah proses konversi oleh Rector menghasilkan jumlah temuan yang identik dengan tahap *pre-scan*, yaitu 11 temuan. Nilai perubahan temuan keamanan ($\Delta F = F_{post} - F_{pre} = 0$) menunjukkan tidak adanya perbedaan. Distribusi jenis kerentanan juga tetap sama, yaitu 1 SQL Injection dan 10 SSRF, seluruhnya berada pada *file* yang sama.

Hasil ini sesuai dengan ekspektasi dan mengindikasikan dua hal. Pertama, proses konversi oleh Rector tidak memperkenalkan kerentanan baru pada kode FT UGM. Kedua,

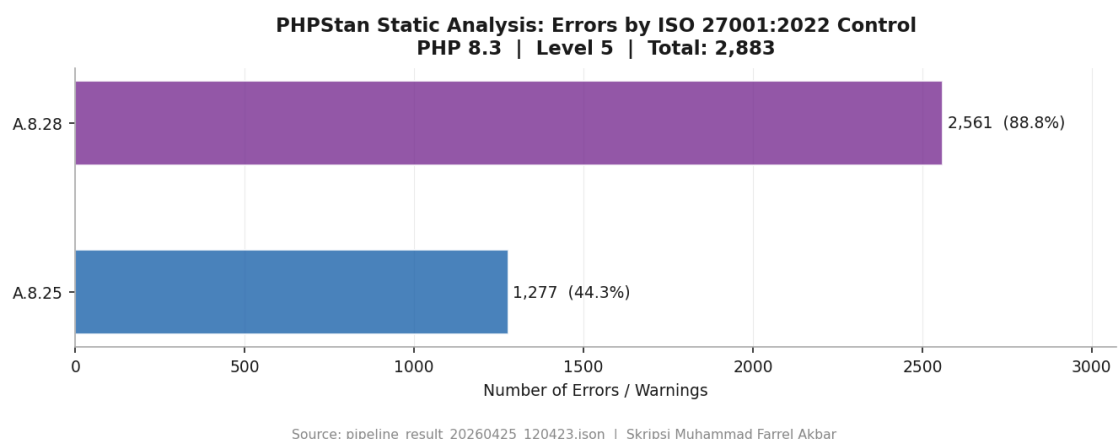
kerentanan yang telah ada sebelumnya tidak dapat diatasi melalui transformasi sintaksis berbasis AST, karena memerlukan perbaikan pada tingkat logika program (semantik), yang berada di luar cakupan pendekatan tersebut.

4.3 Hasil Validasi PHPStan

PHPStan level 5 dijalankan terhadap kode hasil konversi di direktori `output/` dengan target PHP 8.3. Hasilnya adalah 2.871 temuan yang terdiri dari 2.549 ERROR dan 322 WARNING, tersebar di 174 dari 241 *file* yang dianalisis.

Sebagian besar *error* yang ditemukan PHPStan berasal dari tiga kategori utama. Pertama, kelas dari *library* pihak ketiga yang tidak ditemukan, seperti `PhpOffice\PhpSpreadsheet` dan `Dotenv\Dotenv`: PHPStan tidak dapat menemukan kelas-kelas ini karena hanya kode sumber yang dianalisis, bukan dependensi lengkap aplikasi. Kedua, penggunaan nilai kembalian fungsi `void` sebagai nilai, misalnya `Result of method sendResponse() (void) is used`, yang merupakan pola umum pada kode warisan yang ditulis tanpa deklarasi tipe eksplisit. Ketiga, ketidakkonsistenan tipe seperti `Cannot access property $role on array`, yang mencerminkan penggunaan *array* sebagai pengganti objek bertipe tanpa definisi yang jelas.

Semua ini adalah manifestasi (*technical debt*: akumulasi masalah kualitas kode yang ditunda penanganannya) yang sudah ada sebelum migrasi, bukan kesalahan yang diperkenalkan oleh proses konversi. PHP 8.3 memiliki sistem tipe yang jauh lebih ketat dibandingkan PHP 7.x, sehingga masalah yang sebelumnya tidak terdeteksi karena PHP 7.x lebih permisif kini menjadi terlihat oleh PHPStan. Gambar 4.5 menunjukkan distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 yang dipetakan.



Gambar 4.5. Distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 Annex A

Angka PHPStan yang tinggi justru menunjukkan manfaat nyata dari komponen validasi dalam *pipeline* ini. Tanpa PHPStan, 2.871 potensi masalah tipe dan

kompatibilitas ini tidak akan terdeteksi sampai kode dijalankan di lingkungan produksi, sesuai dengan temuan Strobl et al. [11] tentang pentingnya jaminan kualitas otomatis dalam transformasi kode skala besar.

4.4 Status Kepatuhan ISO/IEC 27001:2022

Berdasarkan pemetaan yang dilakukan oleh `iso_mapper.py`, status kepatuhan terhadap enam kontrol ISO/IEC 27001:2022 yang dipetakan adalah sebagaimana ditunjukkan pada Tabel 4.1.

Tabel 4.1. Status kepatuhan ISO/IEC 27001:2022 hasil *pipeline* pada dataset FT UGM

| Kontrol | Judul | Dasar Penilaian | Status |
|---------|--|---|---------------|
| A.5.17 | <i>Authentication Information</i> | Tidak ada temuan <i>hardcoded credentials</i> | COMPLIANT |
| A.8.24 | <i>Use of Cryptography</i> | Tidak ada temuan penggunaan <code>md5()</code> / <code>sha1()</code> untuk kata sandi | COMPLIANT |
| A.8.25 | <i>Secure Development Lifecycle</i> | 1.266 temuan PHPStan (masalah kompatibilitas dan kualitas kode) | NON_COMPLIANT |
| A.8.26 | <i>Application Security Requirements</i> | 1 temuan SQL Injection dari Semgrep | NON_COMPLIANT |
| A.8.28 | <i>Secure Coding</i> | Temuan Semgrep aktif dan 2.549 ERROR PHPStan | NON_COMPLIANT |
| A.8.29 | <i>Security Testing in Development</i> | 10 temuan SSRF dari Semgrep | PARTIAL |

Status keseluruhan adalah NON_COMPLIANT. Ini bukan hasil yang mengejutkan mengingat kondisi kode warisan yang dianalisis; justru ini adalah gambaran yang jujur tentang postur keamanan aplikasi sebelum dilakukan remediasi manual.

Dua kontrol yang berstatus COMPLIANT, yaitu A.5.17 dan A.8.24, perlu dipahami dalam konteks yang tepat. Status ini berarti *ruleset* Semgrep yang digunakan tidak menghasilkan temuan yang dipetakan ke kedua kontrol tersebut, bukan berarti kode bebas dari masalah autentikasi atau kriptografi secara absolut.

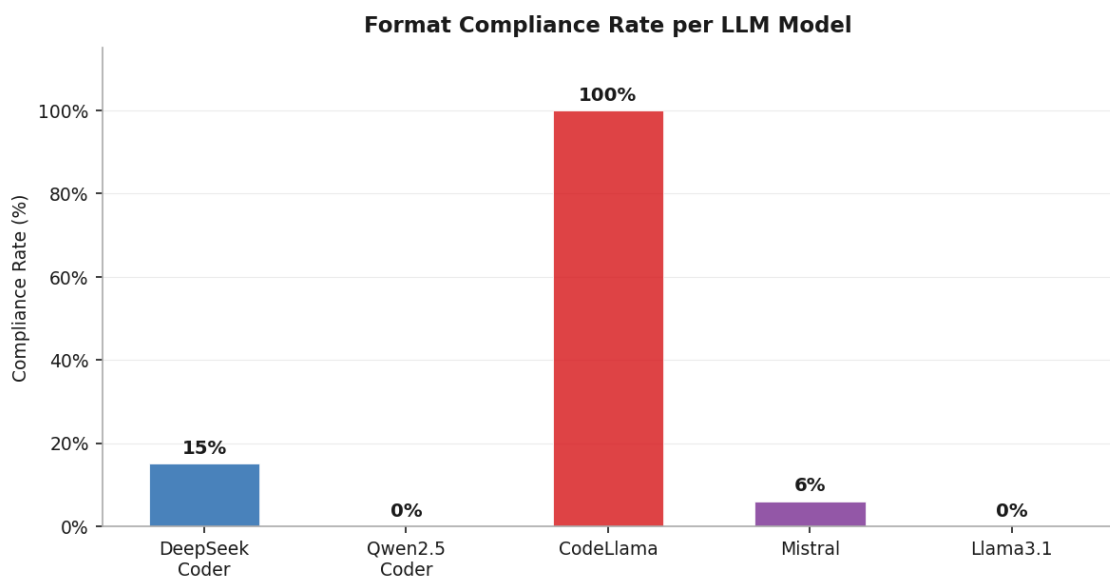
Kontrol A.8.29 berstatus PARTIAL karena *pipeline* sudah menjalankan pengujian keamanan otomatis sebagai bagian dari proses, yang memenuhi sebagian persyaratan kontrol ini. Namun masih ada temuan terbuka, sehingga kontrol belum sepenuhnya terpenuhi. Laporan kepatuhan ini dihasilkan secara otomatis dan dapat langsung digunakan sebagai dokumentasi audit ISO/IEC 27001:2022 [12].

4.5 Hasil Komparasi Lima Model LLM (Kondisi A)

Eksperimen Kondisi A dijalankan dengan *prompt* yang identik untuk semua model (*zero-shot*), 11 temuan Semgrep *pre-scan* sebagai masukan, 3 *run* per temuan per model, dan *temperature* 0,1. Total percobaan adalah 165 (33 per model). Semua model dijalankan secara lokal melalui Ollama tanpa modifikasi.

4.5.1 Format Compliance Rate

Gambar 4.6 menunjukkan *format compliance rate* untuk masing-masing model pada Kondisi A.



Source: llm_comparison_20260425_203514.json | Skripsi Muhammad Farrel Akbar

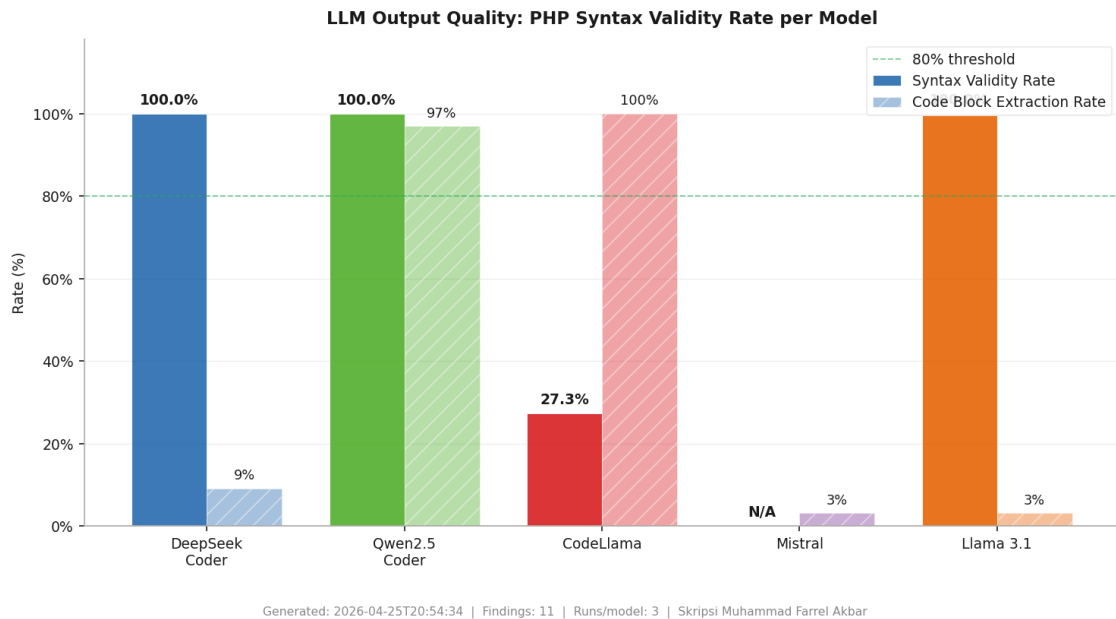
Gambar 4.6. *Format compliance rate* kelima model LLM pada Kondisi A (*zero-shot*, *prompt* identik)

CodeLlama adalah satu-satunya model yang secara konsisten mengikuti format instruksi *prompt* di semua 33 percobaan (100%). DeepSeek Coder mengikuti format

di 5 dari 33 percobaan (15%). Mistral mengikuti format di 2 dari 33 percobaan (6%). Qwen2.5-Coder dan Llama 3.1 tidak menghasilkan satu pun respons yang memenuhi ketiga elemen format sekaligus.

4.5.2 Syntax Validity Rate

Gambar 4.7 menunjukkan *syntax validity rate* dan *code block extraction rate* untuk masing-masing model.



Gambar 4.7. *Syntax validity rate* dan *code block extraction rate* kelima model LLM pada Kondisi A

Hasilnya memperlihatkan gambaran yang berlawanan dengan metrik format. CodeLlama yang memiliki *format compliance* 100% ternyata hanya menghasilkan kode PHP yang *valid* secara sintaksis pada 9 dari 33 blok kode yang berhasil diekstraksi ($SVR = 27,3\%$). Ini berarti 24 dari 33 blok kode CodeLlama mengandung *parse error*.

Sebaliknya, Qwen2.5-Coder yang memiliki *format compliance* 0% berhasil mengekstraksi blok kode PHP dari 32 dari 33 percobaan (97%), dan seluruh 32 blok kode tersebut lolos pemeriksaan `php -l` ($SVR = 100\%$). Model ini mengabaikan instruksi format sepenuhnya, tetapi secara konsisten menghasilkan kode yang berkualitas baik.

Mistral berhasil mengekstraksi 1 blok kode dari 33 percobaan, tetapi blok tersebut tidak lolos `php -l` (0% validitas). DeepSeek Coder dan Llama 3.1 masing-masing mengekstraksi 3 dan 1 blok dengan validitas 100%, namun jumlah sampel yang terlalu kecil membuat angka ini tidak representatif secara statistik.

4.5.3 Statistik Ringkasan

Gambar 4.8 merangkum statistik lengkap untuk kelima model pada Kondisi A.

| LLM Model Comparison -- Confidence Score Statistics | | | | | | |
|---|------|---------|------|------|--------|--------------|
| Model | Mean | Std Dev | Min | Max | Fmt OK | Avg Time (s) |
| DeepSeek
Coder 6.7b | 0.13 | 0.31 | 0.00 | 0.90 | 15% | 4.4 |
| Qwen2.5
Coder 7b | 0.00 | 0.00 | 0.00 | 0.00 | 0% | 10.0 |
| CodeLlama
7b | 0.91 | 0.06 | 0.85 | 1.00 | 100% | 5.0 |
| Mistral
7b | 0.06 | 0.23 | 0.00 | 0.95 | 6% | 7.3 |
| Llama 3.1
8b | 0.00 | 0.00 | 0.00 | 0.00 | 0% | 8.2 |

Source: llm_comparison_20260425_203514.json | Skripsi Muhammad Farrel Akbar

Gambar 4.8. Statistik lengkap komparasi kelima model LLM pada Kondisi A

CodeLlama memiliki *confidence score* rata-rata $\bar{C}_m = 0,909$ (0,909) dengan standar deviasi yang relatif kecil $\sigma_m = 0,054$ (0,054), mencerminkan konsistensi yang tinggi. Namun konsistensi ini tidak berkorelasi dengan kualitas kode: 72,7% kode CodeLlama tidak *valid* secara sintaksis. Ini mengkonfirmasi bahwa *confidence score self-reported* bukan ukuran kualitas kode.

Qwen2.5-Coder memiliki *confidence score* rata-rata $\bar{C}_m = 0,000$ (0,000) karena tidak pernah menghasilkan baris `CONFIDENCE: N`. Ketidakhadiran skor ini bukan tanda ketidakmampuan, melainkan refleksi dari gaya respons model yang mengabaikan instruksi format sambil tetap mengerjakan tugas utama dengan sangat baik. Waktu inferensi Qwen2.5-Coder (10,0 detik) paling lama dibanding model lain, kemungkinan karena model ini menghasilkan respons yang lebih panjang dan detail.

4.5.4 Analisis Kualitatif: Perbandingan Respons terhadap Temuan SQL Injection

Untuk memperlihatkan perbedaan perilaku model secara konkret, berikut perbandingan respons CodeLlama dan Qwen2.5-Coder terhadap temuan SQL Injection yang sama dari `Output.php` baris 768. Kode yang dikirimkan ke model adalah sebagai berikut:

```
$uri .= '?' . $_SERVER['QUERY_STRING'];
$cache_path .= md5(
    $CI->config->item('base_url')
    . $CI->config->item('index_page')
    . ltrim($uri, '/') );
```



```

if ( ! @unlink($cache_path)) {
    log_message('error',
        'Unable to delete cache file for '.$uri);
    return FALSE;
}

```

Respons CodeLlama:

VULNERABILITY: SQL Injection Vulnerability

The vulnerability is caused by the manual construction of an SQL string, which allows user-controlled data to be incorporated into the query. ...

SECURE PHP 8.x REPLACEMENT:

```

```php
$uri = $_SERVER['QUERY_STRING'];
$cache_path = md5(
 $CI->config->item('base_url')
 . $CI->config->item('index_page')
 . ltrim($uri, '/'));
if (! @unlink($cache_path)) {
 log_message('error',
 'Unable to delete cache file for '.$uri);
 return FALSE;
}
(blok kode tidak ditutup, php -l gagal)

```

### **Respons Qwen2.5-Coder:**

### Explanation of Vulnerability

...

Here's the corrected PHP 8.x code:

```

```php
$uri .= '?' . urlencode($_SERVER['QUERY_STRING']);
$cache_path .= md5(
    $CI->config->item('base_url')
    . $CI->config->item('index_page')
    . ltrim($uri, '/'));

if (!@unlink($cache_path)) {
    log_message('error',

```

```

        'Unable to delete cache file for '
        . urlencode($uri));
    return FALSE;
}
'''

```

Dari perbandingan ini terlihat beberapa perbedaan yang signifikan. CodeLlama mengidentifikasi kerentanan dengan benar tapi solusinya tidak tepat: model hanya mengubah cara penulisan kode tanpa benar-benar mengatasi masalahnya. Kode yang dihasilkan masih menggunakan `$_SERVER['QUERY_STRING']` secara langsung tanpa sanitasi apapun.

Qwen2.5-Coder menunjukkan pemahaman kontekstual yang lebih dalam. Model ini menyadari bahwa meskipun Semgrep menandainya sebagai SQL Injection, kode tersebut sebenarnya membangun *cache path*, bukan kueri SQL. Saran perbaikannya menggunakan `urlencode()` untuk menghindari karakter berbahaya dalam *path*. Secara semantik ini lebih tepat, meski solusi yang ideal tetap perlu ditinjau oleh pengembang yang memahami konteks penuh aplikasi.

4.5.5 Trade-off dan Implikasinya

Dari keseluruhan hasil Kondisi A, terdapat *trade-off* yang jelas antara kepatuhan format dan kualitas kode. Model yang paling patuh pada instruksi format (CodeLlama, 100%) menghasilkan kode dengan tingkat sintaksis *error* tertinggi (72,7% kode tidak valid). Model yang menghasilkan kode paling berkualitas (Qwen2.5-Coder, 97% *extraction rate* dan 100% validitas) mengabaikan instruksi format sepenuhnya.

Temuan ini memiliki implikasi langsung untuk desain sistem berbasis LLM lokal: metrik kualitas kode (*syntax validity rate*) adalah indikator yang jauh lebih relevan dibandingkan metrik kepatuhan format. Sistem evaluasi yang hanya mengandalkan kepatuhan format akan salah menilai model mana yang lebih berguna secara praktis.

Trade-off ini juga menunjukkan bahwa kondisi *zero-shot* dengan *prompt* identik mungkin tidak optimal untuk semua model secara bersamaan. Ini menjadi motivasi utama untuk Kondisi B, di mana setiap model menerima *prompt* yang dioptimasi berdasarkan karakteristik dan kelemahan yang terobservasi pada Kondisi A.

4.6 Hasil Komparasi Lima Model LLM (Kondisi B)

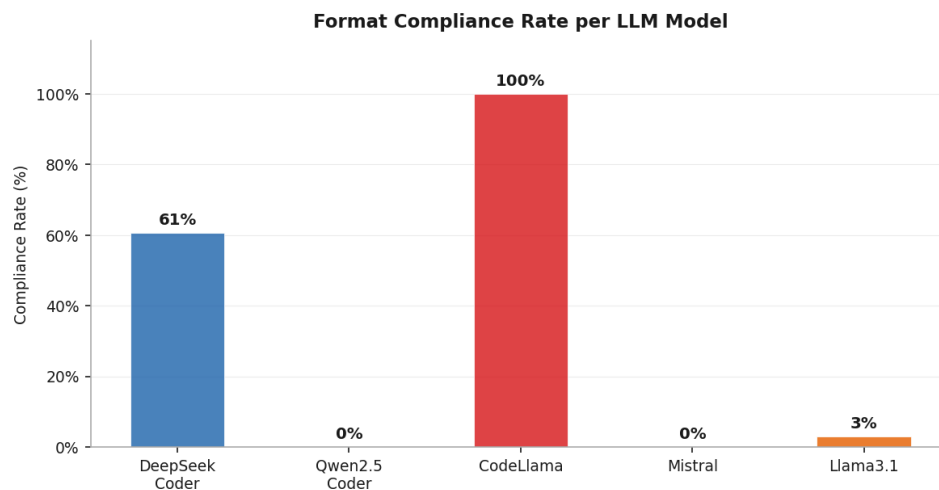
Kondisi B dirancang untuk mengatasi kelemahan spesifik setiap model yang teridentifikasi pada Kondisi A. Setiap model menerima *prompt* yang berbeda melalui metode `_build_optimized_chat_messages()` di `ai_engine.py`, sementara

parameter eksperimen lainnya tetap sama: 11 temuan Semgrep yang sama, 3 *run* per temuan per model, dan *temperature* 0,1.

Strategi optimasi per model dirumuskan berdasarkan pola kegagalan Kondisi A. Qwen2.5-Coder menerima *prompt* yang lebih longgar tanpa tuntutan format ketat karena kualitas kodenya sudah baik. DeepSeek Coder menerima instruksi yang lebih pendek dan langsung untuk mengurangi kompleksitas yang diduga menjadi penyebab rendahnya *extraction rate*. Mistral menerima satu contoh *few-shot* konkret untuk menunjukkan format keluaran yang diharapkan. Llama 3.1 menerima instruksi berbasis *role-playing* eksplisit dengan langkah-langkah bernomor. CodeLlama mempertahankan format Kondisi A dengan tambahan instruksi eksplisit agar menghasilkan kode PHP yang *valid* secara sintaksis.

4.6.1 Format Compliance Rate

Gambar 4.9 menunjukkan *format compliance rate* untuk masing-masing model pada Kondisi B.



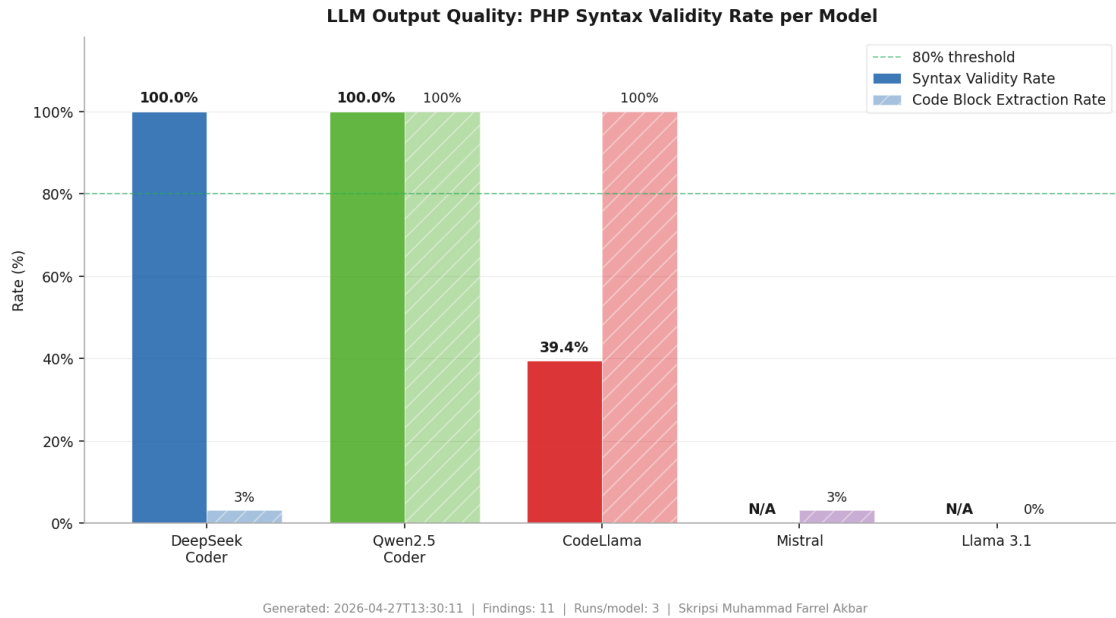
Source: llm_comparison_20260427_130844.json | Skripsi Muhammad Farrel Akbar

Gambar 4.9. *Format compliance rate* kelima model LLM pada Kondisi B (*prompt* dioptimasi per model)

DeepSeek Coder mencatat peningkatan yang paling signifikan, dari 15% pada Kondisi A menjadi 61% pada Kondisi B. Ini mengkonfirmasi bahwa model ini sangat sensitif terhadap kompleksitas instruksi: *prompt* yang lebih pendek dan langsung secara substansial meningkatkan kemampuannya mengikuti format. CodeLlama mempertahankan *format compliance* 100% seperti pada Kondisi A. Sebaliknya, Mistral mengalami regresi dari 6% menjadi 0%, dan Llama 3.1 hanya naik tipis dari 0% menjadi 3% (1 dari 33 *run*).

4.6.2 Syntax Validity Rate

Gambar 4.10 menunjukkan *syntax validity rate* dan *code block extraction rate* pada Kondisi B.



Gambar 4.10. *Syntax validity rate* dan *code block extraction rate* kelima model LLM pada Kondisi B

Qwen2.5-Coder mempertahankan posisinya sebagai model dengan kualitas kode terbaik: *extraction rate* naik dari 97% menjadi 100% dan *syntax validity* tetap 100%. CodeLlama menunjukkan perbaikan parsial pada *syntax validity*, dari 27,3% menjadi 39,4%, meskipun masih jauh dari ambang batas yang dapat diandalkan. DeepSeek Coder mengalami penurunan *extraction rate* dari 9% menjadi 3% meskipun *format compliance*-nya meningkat, yang menunjukkan adanya *trade-off* antara kepatuhan format dan kemampuan menghasilkan blok kode. Mistral dan Llama 3.1 secara efektif tidak menghasilkan kode yang dapat digunakan pada Kondisi B.

4.6.3 Statistik Ringkasan

Gambar 4.11 merangkum statistik lengkap untuk kelima model pada Kondisi B.

LLM Model Comparison -- Confidence Score Statistics						
Model	Mean	Std Dev	Min	Max	Fmt OK	Avg Time (s)
DeepSeek Coder 6.7b	0.59	0.48	0.00	1.00	61%	8.1
Qwen2.5 Coder 7b	0.00	0.00	0.00	0.00	0%	9.7
CodeLlama 7b	0.89	0.16	0.00	1.00	100%	5.6
Mistral 7b	0.00	0.00	0.00	0.00	0%	7.0
Llama 3.1 8b	0.02	0.14	0.00	0.80	3%	8.3

Source: llm_comparison_20260427_130844.json | Skripsi Muhammad Farrel Akbar

Gambar 4.11. Statistik lengkap komparasi kelima model LLM pada Kondisi B

DeepSeek Coder mencatat peningkatan *mean confidence* yang paling mencolok, dari $\bar{C}_m = 0,132$ pada Kondisi A menjadi $\bar{C}_m = 0,588$ pada Kondisi B. Variansi yang tinggi ($\sigma_m = 0,477$) menunjukkan bahwa skor ini tidak stabil: sebagian *run* menghasilkan skor mendekati 1,0 sementara sebagian lainnya tetap di 0,0. Pola ini konsisten dengan *format compliance* 61%: hanya sekitar dua pertiga percobaan yang benar-benar menulis baris `CONFIDENCE :`, sedangkan sisanya tidak sama sekali.

CodeLlama mempertahankan *mean confidence* yang tinggi ($\bar{C}_m = 0,894$) dengan standar deviasi yang kecil ($\sigma_m = 0,162$), mencerminkan konsistensi yang sama seperti pada Kondisi A. Angka ini mungkin tampak meyakinkan, tetapi perlu diingat bahwa *syntax validity rate* CodeLlama hanya 39,4%: artinya model ini sangat yakin dengan jawabannya meski lebih dari separuh kode yang dihasilkannya tidak lolos `php -l`. Ini kembali mengkonfirmasi bahwa *confidence score self-reported* tidak bisa dijadikan proksi kualitas kode.

Qwen2.5-Coder, Mistral, dan Llama 3.1 mencatat *mean confidence* 0,000 karena tidak satu pun respons mereka yang menulis baris `CONFIDENCE :` dalam format yang dapat di-*parse*. Khusus untuk Qwen2.5-Coder, ketiadaan skor ini justru tidak mencerminkan kualitas yang buruk: model ini tetap menghasilkan kode dengan *syntax validity* 100% di Kondisi B, sama seperti Kondisi A. Ketidadaan skor kepercayaan semata-mata mencerminkan gaya respons model yang mengabaikan instruksi format sambil tetap mengerjakan tugas utamanya dengan baik.

Secara keseluruhan, pola yang sama dengan Kondisi A terulang: tidak ada korelasi yang dapat diandalkan antara tingginya *confidence score* dan kualitas kode yang dihasilkan. Ini memperkuat argumen bahwa untuk sistem berbasis LLM lokal berukuran

kecil hingga menengah, *syntax validity rate* tetap menjadi metrik evaluasi yang lebih sah dibandingkan *confidence score self-reported*.

4.7 Perbandingan Kondisi A dan Kondisi B

Tabel 4.2 merangkum perbandingan metrik utama antara Kondisi A dan Kondisi B untuk kelima model.

Tabel 4.2. Perbandingan metrik utama Kondisi A vs Kondisi B untuk kelima model LLM

Model	Format Compliance		Extraction Rate		Syntax Validity	
	Kond. A	Kond. B	Kond. A	Kond. B	Kond. A	Kond. B
DeepSeek Coder 6.7b	15%	61% ↑	9%	3% ↓	100%	100%
Qwen2.5-Coder 7b	0%	0%	97%	100% ↑	100%	100%
CodeLlama 7b	100%	100%	100%	100%	27%	39% ↑
Mistral 7b	6%	0% ↓	3%	3%	0%	0%
Llama 3.1 8b	0%	3% ↑	3%	0% ↓	100%	0% ↓

Dari perbandingan ini, ada tiga pola utama yang dapat diidentifikasi.

Pertama, optimasi *prompt* hanya bekerja signifikan pada model yang memiliki kapabilitas *instruction-following* dasar yang memadai. DeepSeek Coder adalah satu-satunya model yang menunjukkan peningkatan substansial pada *format compliance* (dari 15% menjadi 61%), mengkonfirmasi bahwa model ini peka terhadap kompleksitas instruksi: bukan tidak mampu mengikuti format, tapi terbebani oleh instruksi yang terlalu berlapis pada Kondisi A.

Kedua, strategi *few-shot* untuk Mistral memberikan hasil yang berlawanan dari yang diharapkan: *format compliance* justru turun dari 6% menjadi 0%. Fenomena ini, di mana contoh *few-shot* justru mengalihkan model dari mengerjakan tugas aslinya, konsisten dengan temuan dalam literatur yang menunjukkan bahwa teknik *prompt engineering* yang efektif untuk model besar tidak selalu dapat diterapkan langsung pada model berukuran kecil hingga menengah [13].

Ketiga, keterbatasan CodeLlama dalam menghasilkan kode PHP yang *valid* secara sintaksis bersifat struktural, bukan instruksional. Meskipun instruksi eksplisit untuk memverifikasi sintaks sebelum menulis kode memberikan perbaikan parsial (dari 27% menjadi 39%), sebagian besar kode yang dihasilkan masih mengandung *parse error*. Ini menunjukkan bahwa masalah ini berakar pada representasi internal model terhadap sintaks PHP 8.x, bukan pada cara instruksi diformulasikan.

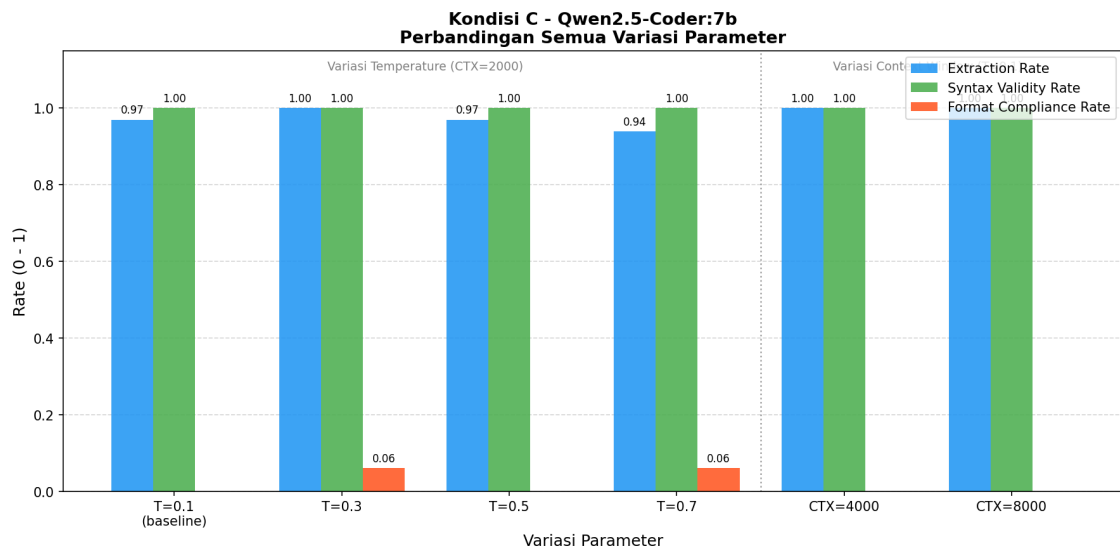
Secara keseluruhan, hasil Kondisi B mempertegas kesimpulan dari Kondisi A bahwa pilihan model adalah faktor yang lebih menentukan dibandingkan strategi *prompt* dalam konteks analisis keamanan kode PHP menggunakan LLM lokal berukuran kecil

hingga menengah. Qwen2.5-Coder secara konsisten menghasilkan kode yang paling berkualitas di kedua kondisi, terlepas dari format *prompt* yang diberikan.

4.8 Optimasi Parameter Qwen2.5-Coder (Kondisi C)

Setelah Kondisi A dan B mengidentifikasi Qwen2.5-Coder sebagai model dengan kualitas kode terbaik, Kondisi C mengevaluasi apakah performa model ini dapat ditingkatkan lebih lanjut melalui variasi dua parameter inferensi: *temperature* dan *context window*. Eksperimen ini menggunakan input, jumlah *run*, dan *prompt* yang identik dengan Kondisi A, hanya parameter yang diuji yang berbeda.

Gambar 4.12 menunjukkan perbandingan ketiga metrik utama di semua variasi parameter.



Gambar 4.12. Perbandingan metrik Qwen2.5-Coder pada variasi *temperature* dan *context window* (Kondisi C)

4.8.1 Pengaruh Temperature

Pada variasi *temperature*, *syntax validity rate* bertahan di 100% di semua nilai yang diuji (0,1 hingga 0,7). Ini menunjukkan bahwa kemampuan Qwen2.5-Coder menghasilkan kode PHP yang *valid* secara sintaksis tidak terdegradasi meski *temperature* dinaikkan secara signifikan.

Extraction rate justru sedikit menurun seiring naiknya *temperature*: dari 100% di T=0,3 menjadi 94% di T=0,7. Ini konsisten dengan ekspektasi teoritis bahwa *temperature* tinggi meningkatkan variabilitas respons, sehingga sebagian kecil respons tidak lagi mengandung blok kode yang dapat diekstraksi.

Format compliance memperlihatkan pola yang tidak monoton: muncul di T=0,3 (6%) dan T=0,7 (6%), tetapi kembali ke 0% di T=0,5. Kemunculan *format compliance*

yang tidak konsisten ini mengindikasikan bahwa kepatuhan format pada Qwen2.5-Coder lebih dipengaruhi oleh variabilitas stokastik daripada perubahan *temperature* secara sistematis.

4.8.2 Pengaruh Context Window

Pada variasi *context window*, baik CTX=4000 maupun CTX=8000 menghasilkan *extraction rate* dan *syntax validity rate* 100%, sedikit di atas baseline CTX=2000 yang memiliki *extraction rate* 97%. *Format compliance* tetap 0% di kedua variasi.

Peningkatan *extraction rate* dari 97% ke 100% pada konteks yang lebih besar mengindikasikan bahwa dengan informasi kode yang lebih lengkap, model lebih konsisten menghasilkan blok kode dalam setiap respons. Namun peningkatan ini marginal dan tidak mengubah karakteristik utama model.

4.8.3 Implikasi Kondisi C

Kondisi C menunjukkan bahwa karakteristik Qwen2.5-Coder lebih ditentukan oleh arsitektur modelnya daripada konfigurasi parameter. Kualitas kode yang dihasilkan sangat stabil di semua variasi parameter yang diuji. Dalam penerapan nyata, hal ini berarti operator tidak perlu melakukan penyesuaian parameter yang ekstensif untuk mendapatkan keluaran berkualitas dari model ini. Konfigurasi *default* (T=0,1, CTX=2000) sudah memberikan hasil yang hampir optimal, sementara peningkatan *context window* ke 4000 atau 8000 karakter dapat memberikan sedikit keuntungan tambahan pada kode yang lebih panjang.

4.9 Analisis Dependensi Composer (FT UGM)

Modul `composer_analyzer.py` membaca berkas `composer.json` pada direktori *root* repositori FT UGM dan memeriksa kompatibilitas setiap dependensi terhadap PHP 8.x menggunakan data dari Packagist. Terdapat 5 dependensi yang teridentifikasi, dengan hasil sebagaimana ditunjukkan pada Tabel 4.3.

Tabel 4.3. Status kompatibilitas PHP 8.x dependensi *composer* FT UGM

Package	Versi	Status
vlucas/phpdotenv	^5.4	SAFE
phpoffice/phpspreadsheet	^1.12	SAFE
firebase/php-jwt	^6.10	SAFE
ngekoding/google-login	^1.0	MANUAL REVIEW
chriskacerguis/codeigniter-restserver	^3.1	MANUAL REVIEW

Tiga dari lima dependensi (`phpdotenv`, `phpspreadsheet`, `firebase/php-jwt`) berstatus `SAFE`, artinya versi yang dikonfigurasi di `composer.json` sudah deklaratif mendukung PHP 8.x. Dua dependensi lainnya berstatus `MANUAL REVIEW`: `ngekoding/google-login` merupakan paket dengan aktivitas pemeliharaan rendah sehingga kompatibilitas PHP 8.x-nya tidak dapat diverifikasi secara otomatis, sementara `chriskacerguis/codeigniter-restserver` merupakan ekstensi pihak ketiga untuk CodeIgniter 3.x yang diketahui tidak memiliki dukungan resmi untuk PHP 8.x dan perlu diganti atau diperbarui secara manual sebelum deployment ke lingkungan PHP 8.3.

4.10 Diskusi dan Keterbatasan

4.10.1 Keterbatasan Dataset

Dataset yang digunakan terdiri dari satu aplikasi berbasis CodeIgniter 3.x milik FT UGM. Dari 245 *file*, 237 diklasifikasikan sebagai *unknown* oleh Rector karena merupakan *file framework* CodeIgniter yang tidak mengandung pola yang perlu ditransformasi. Hal ini membatasi seberapa jauh hasil penelitian ini bisa digeneralisasi ke jenis aplikasi PHP lain, terutama yang tidak menggunakan *framework* atau yang menggunakan *framework* berbeda.

Dari sisi *input* eksperimen LLM, 10 dari 11 temuan adalah SSRF yang berasal dari satu pola kode yang berulang di `Output.php`. Homogenitas *input* ini berarti eksperimen komparasi pada dasarnya menguji respons model terhadap dua jenis kerentanan dengan variasi terbatas. Hasil komparasi perlu diinterpretasikan dalam konteks keterbatasan ini.

4.10.2 Keterbatasan Teknis Pipeline

AI Engine hanya memproses temuan dengan prioritas 1–3 dan membatasi potongan kode yang dikirim ke model pada 2.000 karakter pertama. Untuk *file* PHP yang panjang, konteks yang diterima model terpotong sehingga saran perbaikan mungkin tidak mempertimbangkan keseluruhan logika fungsi. Sebagaimana terlihat pada contoh respons Qwen2.5-Coder, model dapat membuat inferensi yang tepat tentang konteks kode meski dengan informasi yang terbatas, namun tidak selalu bisa dipastikan.

Selain itu, validasi menggunakan `php -l` hanya memeriksa kebenaran sintaksis, bukan kebenaran semantik. Kode yang lolos `php -l` bisa saja masih mengandung kerentanan yang tidak tertangani atau logika yang tidak ekuivalen dengan kode aslinya. Pengujian fungsional terhadap kode hasil perbaikan AI bukan bagian dari cakupan penelitian ini.

4.10.3 Interpretasi Confidence Score

Tidak ada korelasi yang jelas antara *confidence score* yang dilaporkan model dan kualitas kode yang dihasilkan, baik pada Kondisi A maupun Kondisi B. CodeLlama melaporkan kepercayaan diri rata-rata lebih dari 90% di kedua kondisi, tetapi mayoritas kode yang dihasilkannya tidak *valid* secara sintaksis. Qwen2.5-Coder tidak melaporkan skor sama sekali tetapi menghasilkan kode yang hampir selalu berkualitas baik. Temuan ini konsisten dengan literatur yang menunjukkan bahwa model LLM berukuran kecil umumnya tidak terkalibrasi dengan baik dalam menilai *output* mereka sendiri [13] [15]. Dari ketiga metrik yang diukur *FCR*, *extraction rate*, dan *SVR*, hanya *SVR* yang secara konsisten mencerminkan kegunaan praktis model dalam konteks ini.

4.11 Studi Kasus 2: Sistem CBT DTI UGM

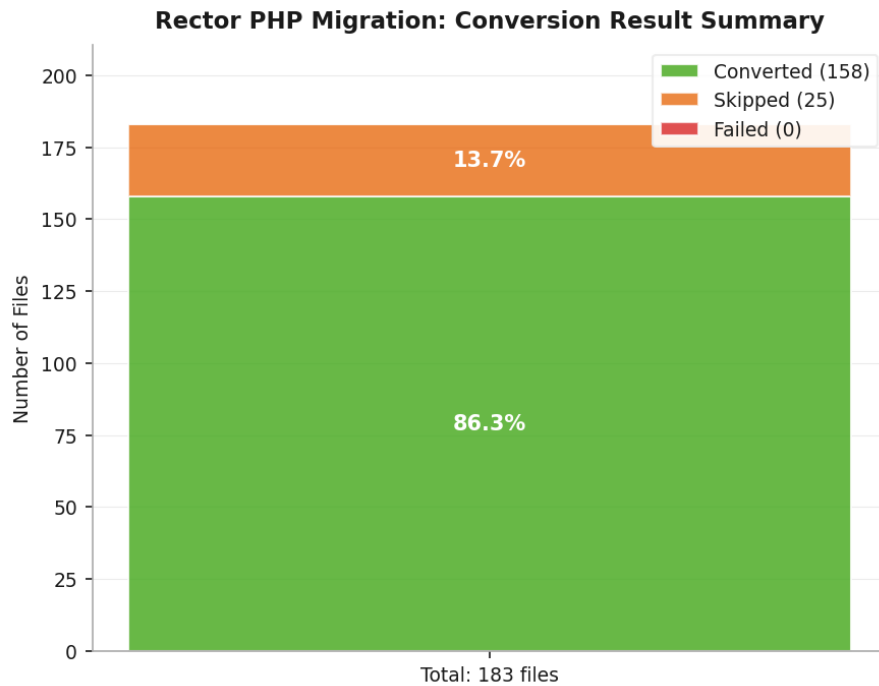
Studi kasus kedua menggunakan kode sumber aplikasi *Computer-Based Test* (CBT) milik Direktorat Teknologi Informasi (DTI) Universitas Gadjah Mada. Aplikasi ini dibangun di atas *framework* CodeIgniter 3 dengan persyaratan versi PHP ≥ 7.0 , dan dikonversi ke target PHP 8.3. *Pipeline* dijalankan terhadap direktori `application/` dengan mengeksklusi subdirektori `libraries/`, `third_party/`, dan `tcpdf/` yang merupakan komponen *framework* dan *library* pihak ketiga, sehingga menghasilkan 183 *file* PHP kustom sebagai masukan.

4.11.1 Hasil Konversi Rector

Rector dijalankan terhadap 183 *file* PHP dengan target versi PHP 8.3. Dari jumlah tersebut, 158 *file* berhasil dikonversi, 25 *file* dilewati (*skipped*), dan tidak ada *file* yang gagal diproses. Tingkat konversi keseluruhan ($CR = 86,3\%$), sebagaimana ditunjukkan pada Gambar 4.13.

Tingkat konversi yang lebih rendah dibanding studi kasus FT UGM (86,3% vs 97,6%) disebabkan oleh proporsi *file* yang dilewati yang lebih besar: 25 dari 183 *file* (13,7%) tidak memerlukan transformasi apa pun karena kodenya sudah kompatibel dengan target PHP 8.3 atau tidak mengandung pola yang perlu ditangani oleh *ruleset* Rector. Tidak adanya *file* yang gagal diproses menunjukkan bahwa kode CBT DTI tidak mengandung konstruksi yang memicu *bug* `FiberNodeScopeResolver` seperti yang ditemukan pada dataset FT UGM.

Seluruh *file* pada direktori `output/` diuji validitas sintaksisnya menggunakan perintah `php -l` dan tidak ditemukan *parse error*, sehingga tingkat validitas sintaks pasca-konversi mencapai 100%.



Source: pipeline_result_20260508_171545.json | Skripsi Muhammad Farrel Akbar

Gambar 4.13. Ringkasan hasil konversi Rector terhadap 183 *file* PHP dataset CBT DTI UGM

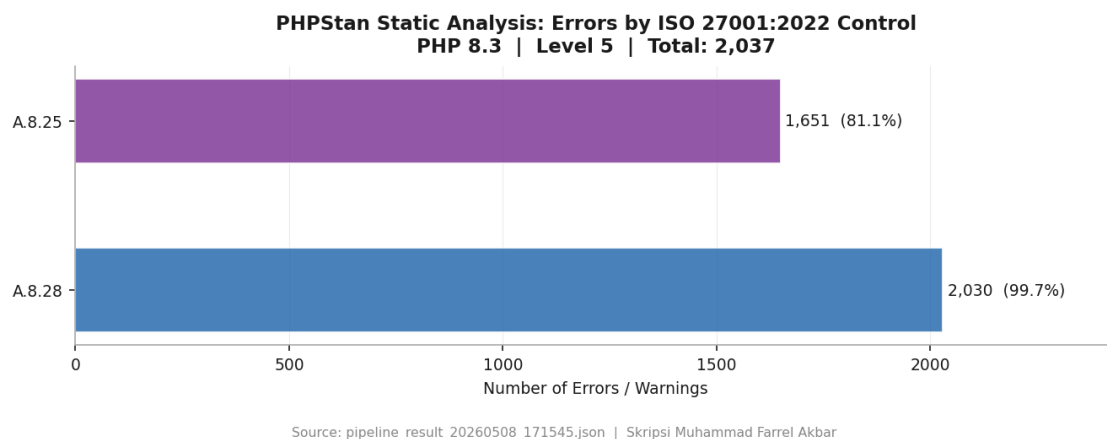
4.11.2 Hasil Pemindaian Keamanan Semgrep

Pemindaian Semgrep terhadap 183 *file* PHP pada direktori `input/` tidak menghasilkan temuan kerentanan (0 *findings*). Pemindaian *post-scan* terhadap direktori `output/` juga menghasilkan 0 temuan, sehingga $\Delta F = 0$.

Tidak adanya temuan Semgrep mengindikasikan bahwa kode kustom CBT DTI tidak mengandung pola kerentanan yang dikenali oleh *ruleset* Semgrep yang digunakan, atau bahwa bagian kode yang berpotensi rentan berada pada komponen *framework* dan *library* yang dieksklusi dari pemindaian. Karena tidak ada temuan yang dapat dikirimkan ke model LLM, modul AI Engine dilewati (*skipped*) secara otomatis oleh *pipeline*.

4.11.3 Hasil Validasi PHPStan

PHPStan *level 5* dijalankan terhadap kode hasil konversi di direktori `output/` dengan target PHP 8.3. Hasilnya adalah 2.037 *error*, seluruhnya berkategori ERROR, tersebar di 97 dari 183 *file* yang dianalisis. Distribusi temuan berdasarkan kontrol ISO/IEC 27001:2022 ditunjukkan pada Gambar 4.14.



Gambar 4.14. Distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 Annex A pada dataset CBT DTI UGM

Sebagaimana pada studi kasus FT UGM, seluruh *error* PHPStan pada dataset CBT DTI merupakan *false positive* yang disebabkan oleh keterbatasan PHPStan dalam memahami arsitektur CodeIgniter 3.x. PHPStan tidak dapat menelusuri dua kategori elemen yang diinjeksi secara dinamis oleh *framework* pada saat *runtime*. Pertama, *magic property*: yaitu properti kelas yang tidak dideklarasikan secara eksplisit dalam kode sumber melainkan diinjeksi melalui metode `__get()` PHP seperti `$this->db` dan `$this->load` yang berasal dari `CI_Controller` dan `CI_Model`. Kedua, fungsi-fungsi *helper* seperti `base_url()`, `redirect()`, dan `form_open()` yang dimuat secara dinamis melalui `$this->load->helper()`. Karena direktori `system/CodeIgniter` dieksklusi dari analisis, PHPStan kehilangan definisi kelas induk dan fungsi-fungsi tersebut sehingga melaporkannya sebagai *undefined*.

Pola distribusi *error* PHPStan pada CBT DTI menunjukkan karakteristik yang berbeda dengan FT UGM: proporsi *error* A.8.25 lebih tinggi (1.651 dari 2.037, atau 81,1%) dibanding FT UGM (1.266 dari 2.871, atau 44,1%). Perbedaan ini mencerminkan komposisi kode yang berbeda: CBT DTI lebih banyak mengandung pola penggunaan *magic property* CI3 yang memicu *error* kategori *Secure Development Lifecycle*, sementara FT UGM memiliki lebih banyak *error* tipe dan kompatibilitas yang terdistribusi di kategori A.8.28.

4.11.4 Status Kepatuhan ISO/IEC 27001:2022

Berdasarkan pemetaan yang dilakukan oleh `iso_mapper.py`, status kepatuhan terhadap enam kontrol ISO/IEC 27001:2022 yang dipetakan adalah sebagaimana ditunjukkan pada Tabel 4.4.

Tabel 4.4. Status kepatuhan ISO/IEC 27001:2022 hasil *pipeline* pada dataset CBT DTI UGM

Kontrol	Judul	Dasar Penilaian	Status
A.5.17	<i>Authentication Information</i>	Tidak ada temuan <i>hardcoded credentials</i>	COMPLIANT
A.8.24	<i>Use of Cryptography</i>	Tidak ada temuan penggunaan <code>md5()</code> / <code>sha1()</code> untuk kata sandi	COMPLIANT
A.8.25	<i>Secure Development Lifecycle</i>	1.651 temuan PHPStan (<i>false positive</i> CI3)	NON_COMPLIANT
A.8.26	<i>Application Security Requirements</i>	Tidak ada temuan SQL Injection dari Semgrep	COMPLIANT
A.8.28	<i>Secure Coding</i>	2.030 dari 2.037 ERROR PHPStan (<i>false positive</i> CI3)	NON_COMPLIANT
A.8.29	<i>Security Testing in Development</i>	Pemindaian Semgrep dijalankan, tidak ada temuan terbuka	COMPLIANT

Status keseluruhan adalah NON_COMPLIANT karena adanya temuan PHPStan pada A.8.25 dan A.8.28. Perlu dicatat bahwa perbedaan status dibanding FT UGM terdapat pada A.8.26 dan A.8.29: kedua kontrol ini berstatus COMPLIANT pada CBT DTI karena tidak ada temuan Semgrep aktif, sementara pada FT UGM keduanya berstatus NON_COMPLIANT dan PARTIAL akibat adanya temuan SQL Injection dan SSRF.

4.11.5 Analisis Dependensi Composer

Aplikasi CBT DTI UGM tidak memiliki berkas `composer.json` pada tingkat aplikasi. Komponen pihak ketiga seperti *library* TCPDF disertakan langsung sebagai direktori dalam repositori dan telah dieksklusi dari analisis *pipeline*. Oleh karena itu, modul `composer_analyzer.py` tidak menghasilkan keluaran untuk studi kasus ini

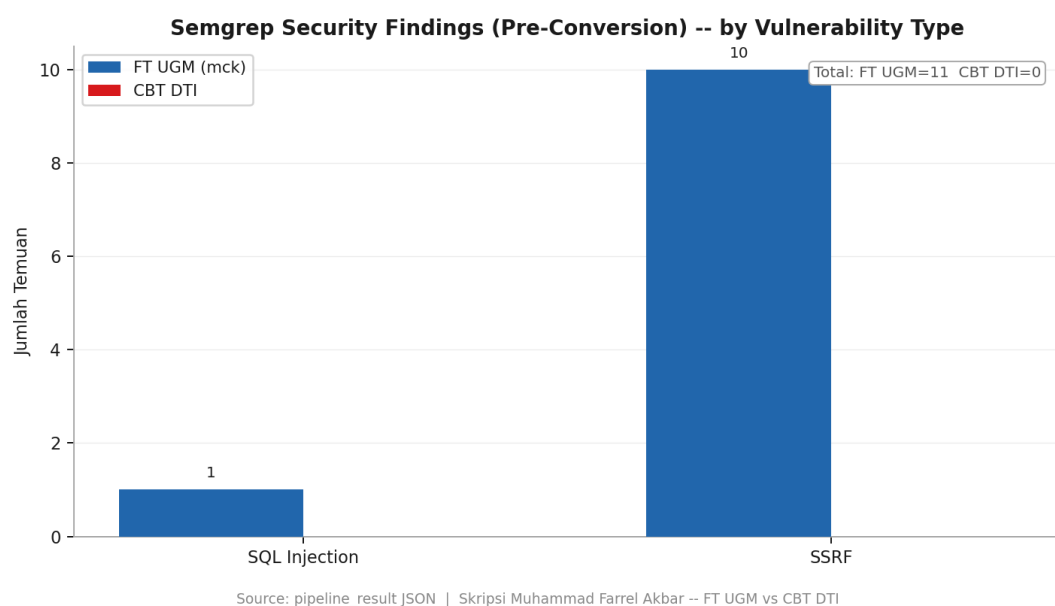
dan langkah analisis dependensi dilewati secara otomatis.

4.12 Perbandingan Kedua Studi Kasus

Bagian ini membandingkan hasil implementasi *pipeline* pada kedua studi kasus secara berdampingan untuk mengidentifikasi pola yang konsisten maupun perbedaan yang signifikan.

4.12.1 Perbandingan Temuan Keamanan Semgrep

Gambar 4.15 menunjukkan perbandingan temuan Semgrep *pre-scan* berdasarkan jenis kerentanan pada kedua dataset.

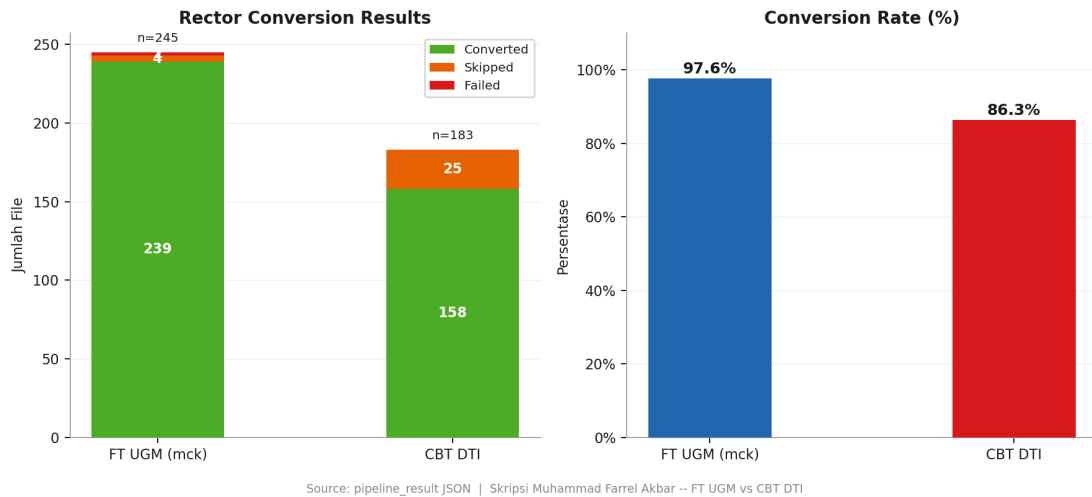


Gambar 4.15. Perbandingan temuan Semgrep *pre-scan* berdasarkan jenis kerentanan pada kedua studi kasus

Dataset FT UGM mengandung 11 temuan aktif (1 SQL Injection dan 10 SSRF), sementara dataset CBT DTI tidak menghasilkan temuan sama sekali. Perbedaan ini tidak serta-merta berarti kode CBT DTI lebih aman secara absolut; melainkan mencerminkan perbedaan arsitektur dan cakupan kode yang dianalisis. Seluruh temuan FT UGM terpusat pada `Output.php`, sebuah modul *framework* yang menangani operasi *cache* dengan interaksi tinggi terhadap data eksternal. Modul setara pada CBT DTI kemungkinan berada di dalam direktori *framework* yang dieksklusi dari pemindaian.

4.12.2 Perbandingan Tingkat Konversi Rector

Gambar 4.16 menunjukkan perbandingan hasil konversi Rector pada kedua dataset.



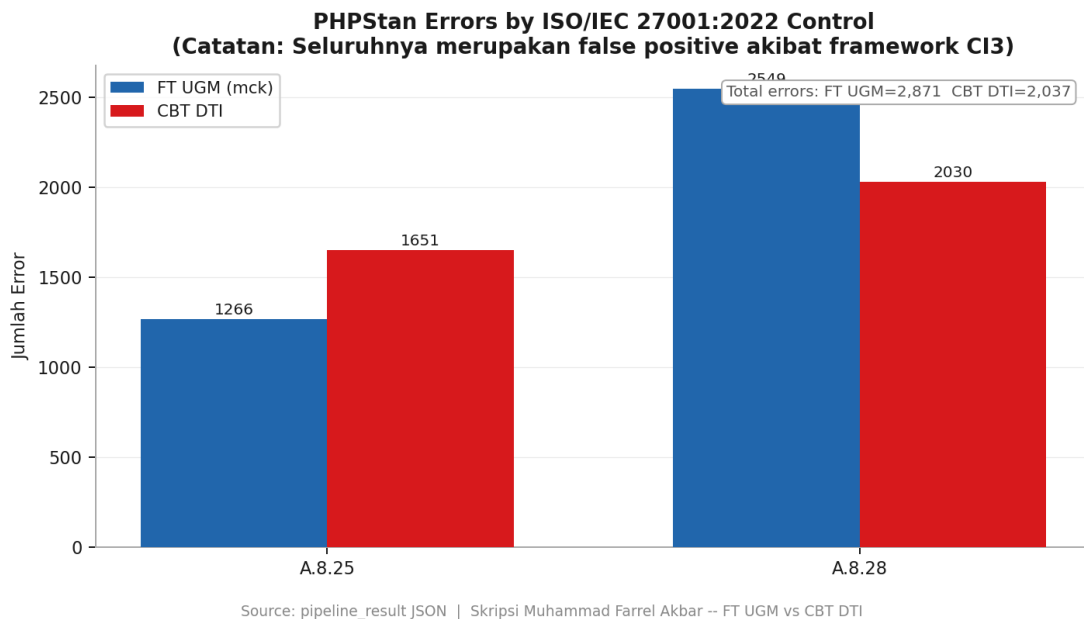
Gambar 4.16. Perbandingan hasil konversi Rector pada kedua studi kasus

FT UGM mencatat *conversion rate* 97,6% (239/245) sementara CBT DTI mencatat 86,3% (158/183). Perbedaan 11,3 poin persentase ini terutama disebabkan oleh proporsi *file* yang dilewati: CBT DTI memiliki 25 *file skipped* (13,7%) dibanding 4 *file* pada FT UGM (1,6%). *File* yang dilewati adalah *file* yang sudah kompatibel dengan PHP 8.3 atau tidak mengandung pola yang perlu ditangani Rector, sehingga *skipped* bukan berarti gagal melainkan sudah kompatibel dengan PHP 8.3. FT UGM juga memiliki 2 *file* yang gagal diproses akibat *bug* `FiberNodeScopeResolver`, sementara CBT DTI tidak mengalami kegagalan yang sama. Pada kedua studi kasus, validitas sintaks pasca-konversi mencapai 100%.

4.12.3 Perbandingan Temuan PHPStan

Gambar 4.17 menunjukkan perbandingan distribusi *error* PHPStan berdasarkan kontrol ISO/IEC 27001:2022 pada kedua dataset.

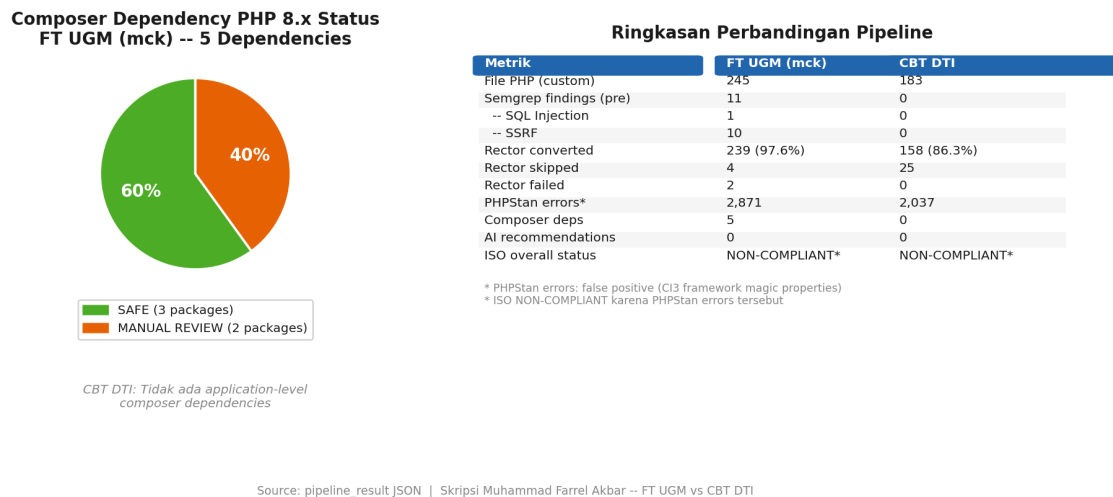
FT UGM menghasilkan 2.871 *error* PHPStan sementara CBT DTI menghasilkan 2.037 *error*. Keduanya dijalankan pada PHPStan *level 5* dengan target PHP 8.3, sehingga angka ini dapat dibandingkan secara langsung. Seluruh *error* pada kedua dataset merupakan *false positive* akibat keterbatasan PHPStan terhadap arsitektur CodeIgniter 3.x sebagaimana dijelaskan pada subbab sebelumnya. Jumlah *error* yang lebih tinggi pada FT UGM sebanding dengan ukuran dataset yang lebih besar (245 vs 183 *file*) dan penggunaan dependensi pihak ketiga yang lebih beragam.



Gambar 4.17. Perbandingan *error* PHPStan berdasarkan kontrol ISO/IEC 27001:2022 pada kedua studi kasus

4.12.4 Ringkasan Perbandingan dan Analisis Dependensi Composer

Gambar 4.18 merangkum seluruh metrik utama kedua studi kasus dalam satu tampilan, termasuk status dependensi *composer* FT UGM.



Gambar 4.18. Ringkasan perbandingan metrik *pipeline* dan status dependensi *composer* kedua studi kasus

Secara keseluruhan, kedua studi kasus menunjukkan pola yang konsisten: *pipeline* berhasil menjalankan seluruh tahapan secara otomatis, konversi Rector menghasilkan kode yang valid secara sintaksis di kedua dataset, dan status ISO/IEC 27001:2022 keduanya adalah NON_COMPLIANT yang disebabkan oleh *error* PHPStan. Perbedaan utama terletak pada keberadaan temuan Semgrep: FT UGM memiliki

kerentanan aktif yang memerlukan remediasi manual, sementara CBT DTI bersih dari temuan Semgrep pada kode kustomnya. Perbedaan lain adalah durasi eksekusi: FT UGM membutuhkan sekitar 41 menit karena eksperimen LLM dijalankan terhadap 11 temuan Semgrep, sementara CBT DTI selesai dalam sekitar 50 detik karena langkah AI Engine dilewati akibat tidak adanya temuan Semgrep.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Penelitian ini membangun *pipeline* berbasis Python yang mengintegrasikan konversi otomatis aplikasi web PHP lama ke versi PHP target dengan pemeriksaan kepatuhan *secure coding* berbasis ISO/IEC 27001:2022 dalam satu alur kerja *open source* yang dapat dijalankan secara lokal. Komponen konversi utama adalah Rector yang bekerja secara deterministik berbasis AST, sementara komponen AI berperan sebagai pemberi saran perbaikan semantik terhadap kerentanan yang tidak dapat ditangani oleh transformasi sintaksis. Berdasarkan hasil implementasi pada dua studi kasus nyata, berikut kesimpulan yang menjawab tujuan penelitian yang telah ditetapkan.

Pertama, platform yang mampu melakukan konversi otomatis aplikasi web PHP lama ke versi PHP target yang dapat dikonfigurasi berhasil dikembangkan. Rector mencatat tingkat konversi 97,6% pada dataset FT UGM (239 dari 245 *file*) dan 86,3% pada dataset CBT DTI UGM (158 dari 183 *file*), dengan tingkat validitas sintaks pasca-konversi mencapai 100% pada kedua studi kasus. Dua *file* yang gagal pada dataset FT UGM disebabkan oleh *bug* internal Rector pada `FiberNodeScopeResolver`, bukan berasal dari kode sumber. *Pipeline* terbukti mampu menangani kode dari berbagai era PHP secara serentak melalui mekanisme *ruleset* kumulatif Rector, dengan durasi eksekusi sekitar 41 menit untuk dataset FT UGM (termasuk eksperimen LLM terhadap 11 temuan) dan sekitar 50 detik untuk dataset CBT DTI UGM (tanpa komponen AI karena tidak ada temuan Semgrep).

Kedua, mekanisme pemeriksaan *secure coding* yang terhubung dengan kontrol ISO/IEC 27001:2022 Annex A berhasil diimplementasikan. Semgrep dijalankan sebelum dan sesudah konversi untuk mengukur perubahan postur keamanan, sementara `iso_mapper.py` memetakan setiap temuan ke kontrol yang relevan secara deterministik dan dapat diaudit. Nilai $\Delta F = 0$ (selisih jumlah temuan keamanan antara *pre-scan* dan *post-scan*) pada kedua studi kasus mengkonfirmasi bahwa proses konversi tidak memperkenalkan kerentanan baru. Status kepatuhan keseluruhan pada kedua dataset adalah `NON_COMPLIANT`, yang mencerminkan kondisi nyata kode warisan sebelum remediasi manual dilakukan, bukan kegagalan *pipeline*.

Ketiga, sistem *quality assurance* yang memeriksa hasil konversi terhadap kebenaran fungsional dan kepatuhan keamanan berhasil dibangun. PHPStan *level* 5 mendeteksi 2.871 temuan pada dataset FT UGM dan 2.037 temuan pada dataset CBT DTI UGM. Berdasarkan analisis kategoris pada Subbab ??, seluruh temuan ini merupakan *false positive* yang disebabkan oleh keterbatasan PHPStan dalam menelusuri

magic property dan fungsi *helper* yang diinjeksi secara dinamis oleh arsitektur CodeIgniter 3.x pada saat *runtime*, bukan akibat proses konversi. Meskipun demikian, angka ini memperlihatkan manfaat nyata komponen validasi: tanpa PHPStan, ribuan potensi masalah kompatibilitas tipe tidak akan terdeteksi sebelum kode dijalankan di lingkungan produksi, sebagaimana ditemukan oleh Strobl et al. [11].

Keempat, dokumentasi otomatis berupa laporan konversi dan pemetaan kepatuhan ISO/IEC 27001:2022 berhasil dihasilkan. Laporan JSON yang dihasilkan secara otomatis mencakup status konversi per *file*, temuan keamanan Semgrep, hasil validasi PHPStan, status per kontrol ISO, dan kompatibilitas dependensi *composer*. Laporan ini dapat langsung digunakan sebagai dokumentasi audit tanpa pekerjaan manual tambahan.

Di luar keempat tujuan tersebut, penelitian ini menghasilkan temuan empiris tentang perilaku model LLM lokal berukuran kecil dalam konteks analisis keamanan kode. Eksperimen komparasi terhadap lima model menunjukkan bahwa *Format Compliance Rate* dan *Syntax Validity Rate* merupakan dua dimensi yang independen: CodeLlama mencatat FCR tertinggi (100%) namun SVR terendah (27,3% pada Kondisi A), sementara Qwen2.5-Coder mengabaikan instruksi format sepenuhnya (FCR 0%) namun menghasilkan kode dengan SVR 100% secara konsisten di semua kondisi eksperimen. Untuk keperluan integrasi ke dalam sistem otomatis, Qwen2.5-Coder direkomendasikan sebagai model utama dengan catatan bahwa mekanisme ekstraksi kode perlu disesuaikan agar tidak bergantung pada kepatuhan format *output*. Temuan ini menunjukkan bahwa SVR adalah metrik evaluasi yang lebih sahih dibandingkan FCR maupun *confidence score self-reported* untuk sistem berbasis LLM lokal berukuran kecil hingga menengah.

5.2 Saran

Berdasarkan keterbatasan yang teridentifikasi selama penelitian, berikut beberapa arah pengembangan yang dapat dilakukan pada penelitian selanjutnya.

Pertama, evaluasi semantik terhadap saran perbaikan AI perlu dikembangkan. Validasi menggunakan `php -l` hanya memeriksa kebenaran sintaksis, bukan kebenaran logika perbaikan. Penelitian selanjutnya dapat membangun *test suite* otomatis atau memanfaatkan eksekusi kode terisolasi untuk memverifikasi bahwa kode yang dihasilkan model benar-benar memperbaiki kerentanan yang ditargetkan, bukan sekadar menghasilkan kode yang dapat dikompilasi.

Kedua, dataset eksperimen LLM perlu diperluas dengan keragaman kerentanan yang lebih tinggi. Pada penelitian ini, 10 dari 11 temuan Semgrep berasal dari satu pola SSRF yang berulang pada satu *file*. Evaluasi yang lebih komprehensif membutuhkan dataset yang mencakup berbagai jenis kerentanan dari banyak *file* berbeda, sehingga

kesimpulan tentang perbandingan model dapat digeneralisasi lebih luas.

Ketiga, penanganan *false positive* PHPStan pada basis kode CodeIgniter 3.x perlu ditingkatkan. Penelitian selanjutnya dapat mengeksplorasi penggunaan *custom stubs* PHPStan untuk mendefinisikan *magic property* dan fungsi *helper* CI3, atau menjalankan PHPStan dengan konfigurasi *baseline* sehingga hanya temuan baru yang diperkenalkan oleh konversi yang dilaporkan. Pendekatan ini akan meningkatkan akurasi laporan kepatuhan ISO pada aplikasi berbasis CodeIgniter.

Keempat, integrasi *pipeline* ke dalam alur kerja CI/CD dapat diujicobakan. Saat ini *pipeline* dijalankan secara manual. Integrasi ke GitHub Actions atau GitLab CI akan memungkinkan pemeriksaan keamanan dan kepatuhan ISO dijalankan secara otomatis pada setiap *commit*, menjadikan *pipeline* ini bagian permanen dari siklus pengembangan perangkat lunak yang berkelanjutan.

Kelima, perbandingan dengan alat analisis keamanan komersial perlu dilakukan untuk memvalidasi temuan secara eksternal. Penelitian ini menggunakan alat *open source* sepenuhnya. Perbandingan hasil Semgrep dan PHPStan dengan alat seperti SonarQube atau Snyk pada dataset yang sama akan memberikan gambaran yang lebih komprehensif tentang cakupan dan akurasi deteksi kerentanan yang dicapai oleh *pipeline* ini.

Keenam, validasi pemetaan ISO/IEC 27001:2022 oleh pakar domain perlu dilakukan. Pemetaan dalam `iso_mapper.py` dibangun berdasarkan interpretasi peneliti terhadap dokumen standar. Keterlibatan auditor atau praktisi bersertifikat ISO/IEC 27001 dalam mengevaluasi kesesuaian setiap pemetaan akan meningkatkan validitas dan kepercayaan laporan kepatuhan yang dihasilkan.

Ketujuh, eksplorasi *fine-tuning* model LLM pada dataset kerentanan kode PHP dapat dilakukan sebagai kelanjutan penelitian ini. Penelitian ini menyediakan *baseline* evaluasi berbasis *prompt engineering* yang dapat dijadikan titik pembandingan apabila pendekatan *fine-tuning* diterapkan di masa depan. Teknik *parameter-efficient fine-tuning* seperti LoRA dan QLoRA membuka kemungkinan adaptasi model pada perangkat keras kelas menengah dengan tetap menjaga privasi data kode sumber institusi.

Kedelapan, perluasan cakupan bahasa pemrograman dan *framework* dapat diujicobakan. Arsitektur modular *pipeline* yang dibangun memungkinkan penggantian komponen Rector dan Semgrep dengan alat setara untuk bahasa lain, sehingga pendekatan yang sama berpotensi diterapkan pada migrasi aplikasi berbasis Python, JavaScript, atau bahasa lain yang menghadapi tantangan serupa.

DAFTAR PUSTAKA

- [1] The PHP Group, “PHP: History of PHP,” php.net, 2024, [Online]. Available: <https://www.php.net/manual/en/history.php>. [Accessed: Apr. 2026].
- [2] W3Techs, “Usage statistics and market share of PHP for websites,” W3Techs Web Technology Surveys, 2026, [Online]. Available: <https://w3techs.com/technologies/details/pl-php>. [Accessed: Apr. 8, 2026].
- [3] The PHP Group, “PHP: Supported versions,” php.net, 2025, [Online]. Available: <https://www.php.net/supported-versions.php>. [Accessed: Apr. 2026].
- [4] National Institute of Standards and Technology, “National vulnerability database (NVD),” NIST, 2024, [Online]. Available: <https://nvd.nist.gov/>. [Accessed: Apr. 2026].
- [5] OWASP Foundation, “OWASP top 10:2021 — the ten most critical web application security risks,” OWASP Foundation, 2021, [Online]. Available: <https://owasp.org/Top10/>. [Accessed: Apr. 2026].
- [6] E. Merlo, D. Letarte, and G. Antoniol, “Automated protection of PHP applications against SQL-injection attacks,” in *Proc. 11th European Conf. on Software Maintenance and Reengineering (CSMR)*, Amsterdam, Netherlands, 2007, pp. 191–202.
- [7] Rector, “Rector — instant upgrades and automated refactoring,” getrector.com, 2024, [Online]. Available: <https://getrector.com>. [Accessed: Apr. 2026].
- [8] D. Chen, D. Lawler, Y. Zhang, P. Kuchhal, D. Westcott, and G. Yang, “AST-based source code migration through symbols replacement,” in *Proc. 2022 IEEE Asia-Pacific Conf. on Computer Science and Data Engineering (CSDE)*, Brisbane, Australia, 2022, pp. 1–6.
- [9] Semgrep, “Semgrep — lightweight static analysis for many languages,” semgrep.dev, 2024, [Online]. Available: <https://semgrep.dev>. [Accessed: Apr. 2026].
- [10] PHPStan, “PHPStan — php static analysis tool,” phpstan.org, 2024, [Online]. Available: <https://phpstan.org>. [Accessed: Apr. 2026].
- [11] S. Strobl, C. Zoffi, C. Haselmann, M. Bernhart, and T. Grechenig, “Automated code transformations: Dealing with the aftermath,” in *Proc. 2020 IEEE 27th Int. Conf. on Software Analysis, Evolution and Reengineering (SANER)*, London, ON, Canada, 2020, pp. 627–631.
- [12] International Organization for Standardization and International Electrotechnical Commission, *ISO/IEC 27001:2022 Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems: Requirements*, International Organization for Standardization and International Electrotechnical Commission Std., 2022.

- [13] V. Akuthota, R. Kasula, S. T. Sumona, M. Mohiuddin, M. T. Reza, and M. M. Rahman, “Vulnerability detection and monitoring using LLM,” in *Proc. 2023 IEEE 9th Int. Women in Engineering (WIE) Conf. on Electrical and Computer Engineering (WIECON-ECE)*, Thiruvananthapuram, India, 2023, pp. 309–314.
- [14] C. Li, B. Guan, Q. Zheng, B. Liu, and Z. Zhang, “Software vulnerability detection based on LLM,” in *Proc. 2025 IEEE 7th Advanced Information Management, Communications, Electronic and Automation Control Conf. (IMCEC)*, Chongqing, China, 2025, pp. 1534–1539.
- [15] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities,” in *Proc. 13th Int. Conf. on Learning Representations (ICLR)*, Singapura, 2025, [Online]. Available: <https://arxiv.org/abs/2405.17238>.
- [16] J. Stümpfle, D. Atray, N. Jazdi, and M. Weyrich, “Large language model assisted transformation of software variants into a software product line,” in *Proc. 2025 IEEE/ACM 22nd Int. Conf. on Software and Systems Reuse (ICSR)*, Ottawa, ON, Canada, 2025, pp. 12–20.
- [17] DeepSeek-AI, “DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence,” *arXiv preprint arXiv:2401.14196*, 2024, [Online]. Available: <https://arxiv.org/abs/2401.14196>.
- [18] B. Hui *et al.*, “Qwen2.5-Coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024, [Online]. Available: <https://arxiv.org/abs/2409.12186>.
- [19] B. Rozière *et al.*, “Code Llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2024, [Online]. Available: <https://arxiv.org/abs/2308.12950>.
- [20] A. Q. Jiang *et al.*, “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023, [Online]. Available: <https://arxiv.org/abs/2310.06825>.
- [21] A. Dubey *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024, [Online]. Available: <https://arxiv.org/abs/2407.21783>.
- [22] Ollama, “Ollama — get up and running with large language models locally,” ollama.com, 2024, [Online]. Available: <https://ollama.com>. [Accessed: Apr. 2026].
- [23] Z. Cai, L. Zhao, X. Wang, X. Yang, J. Qin, and K. Yin, “A pattern-based code transformation approach for cloud application migration,” in *Proc. 2015 IEEE 8th Int. Conf. on Cloud Computing (CLOUD)*, 2015, pp. 33–40.

Catatan: Daftar pustaka adalah apa yang dirujuk atau disitasi, bukan apa yang telah dibaca, jika tidak ada dalam sitasi maka tidak perlu dituliskan dalam daftar pustaka.